

A PROJECT REPORT ON

Intelligent Conversational AI Chatbot for Automated Customer Support

Handling Billing Disputes and Personalized Offer Recommendations using Natural Language Processing, Deep-Learning-based Natural Language Understanding, a Hub-and-Spoke Multi-Agent Architecture, and Reinforcement Learning

Submitted in partial fulfilment of the requirements for the degree of
Master of Computer Applications (MCA)

Project Type: Proof-of-Concept Implementation

Academic Year: 2025–2026

Declaration

I hereby declare that the project work entitled “*Intelligent Conversational AI Chatbot for Automated Customer Support — Handling Billing Disputes and Personalized Offer Recommendations*” submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications is a record of original work carried out by me. The synthetic datasets, models, source code, diagrams and results presented in this report are the outcome of my own implementation effort. Any external concepts, libraries or references used have been duly acknowledged. This work, in whole or in part, has not been submitted previously for the award of any other degree or diploma.

Signature: _____

Name: _____

Date: _____

Place: _____

Certificate

This is to certify that the project report entitled “*Intelligent Conversational AI Chatbot for Automated Customer Support*” is a bona fide record of the project work carried out under my supervision in partial fulfilment of the requirements for the degree of Master of Computer Applications. To the best of my knowledge, the work reported herein is original and has not formed the basis for the award of any earlier degree.

Project Guide: _____ Head of Department: _____

Internal Examiner: _____ External Examiner: _____

Acknowledgement

I take this opportunity to express my sincere gratitude to my project guide for their invaluable guidance, encouragement and continuous support throughout the course of this project. Their insights into machine learning, conversational systems and software engineering were instrumental in shaping this work. I extend my thanks to the Head of the Department and the faculty of the Department of Computer Applications for providing the academic environment and infrastructure that made this project possible. I am also grateful to my peers for their constructive feedback during design reviews and demonstrations. Finally, I thank my family for their unwavering support and patience during the development of this project.

Abstract

Customer-support contact centres in the telecommunications industry handle enormous volumes of repetitive interactions. Analyses of support traffic consistently show that two intents dominate: customers questioning or disputing a charge on their bill, and customers seeking a better plan or a discount. Both categories are latency-sensitive, rule-heavy and emotionally charged, which makes them simultaneously expensive to service with human agents and highly amenable to automation with modern Natural Language Processing (NLP). This project designs and implements an intelligent conversational Artificial Intelligence (AI) chatbot that provides automated, accurate and real-time customer support for exactly these two intents — **billing-dispute resolution** and **personalized offer recommendation**.

The system is built on a “thick brain, thin interface” philosophy in which the user interface performs only presentation while all language understanding, decision-making and action execution reside in a deterministic Python brain. Customer utterances are interpreted by a local, deep-learning-based Natural Language Understanding (NLU) stack comprising a Multi-Layer-Perceptron (MLP) neural-network intent classifier over Term-Frequency–Inverse-Document-Frequency (TF-IDF) features, a second MLP for sentiment analysis, and rule-based entity extraction. These signals are fused into a per-turn frame that drives a **hub-and-spoke multi-agent** dialogue engine: a coordinator (the hub) routes each conversational turn between a front-line billing-and-offers agent and a senior retention specialist, with the two agents never communicating directly. Responses are produced by a Natural Language Generation (NLG) module that pairs grounded response templates with an optional local Large Language Model (LLM, served by Ollama) for human-like phrasing, while strictly forbidding the model from inventing facts. Offer recommendations are ranked by a deterministic scoring engine and continuously improved by a **reinforcement-learning** bandit that learns which offers customers accept. Before any account data is served, the customer must pass a two-step identity validation.

The chatbot is delivered as a runnable Streamlit web application backed by a fully synthetic dataset, and is instrumented with live signals and Key Performance Indicators (KPIs) for monitoring. Evaluation demonstrates a sentiment classification accuracy of approximately 83% and an intent classification accuracy of approximately 62% on a compact held-out set (bounded by the proof-of-concept corpus size), a 100% resolution rate on the designed test scenarios, and low per-turn latency. The project fulfils all five stated objectives and provides a clear, modular foundation for a production system.

Keywords: Conversational AI, Chatbot, Natural Language Processing, Natural Language Understanding, Intent Classification, Sentiment Analysis, Natural Language Generation, Large Language Models, Reinforcement Learning, Recommender Systems, Hub-and-Spoke Architecture, Multi-Agent Systems, Telecom Customer Support, Billing Dispute, Streamlit.

Table of Contents

1. Introduction — Overview, Background, Problem Statement, Motivation, Objectives, Scope, Organization
2. Literature Review and Background — Conversational AI, NLP, NLU, Sentiment, ML, Deep Learning, LLMs, RL, Recommender Systems, Dialogue Management, Related Work, Research Gap
3. System Analysis — Existing System, Proposed System, Feasibility, Functional and Non-Functional Requirements, Use-Case Identification, Hardware/Software Requirements
4. System Design — Design Goals, High-Level Architecture, Subsystem Decomposition, Per-Turn Pipeline, Hub-and-Spoke Design, Data-Flow Diagrams, Data Model, Class Design
5. Methodology by Milestone — Data Collection, Data Preparation, NLU Model, NLG Module, Reinforcement Learning, Deployment and Monitoring, with Outcomes
6. Use Cases in Detail — Identity Validation, Billing Dispute, Offer Recommendation, Retention Handoff, Ticket Status, each with a Sequence Diagram and Explanation
7. Implementation — Codebase Folder Structure, Technology Stack, Module Implementations, Configuration
8. Testing — Testing Approach, Levels of Testing, Test Cases, Test Scenarios, Results
9. Results and Evaluation — NLU Metrics, Conversation KPIs, Screenshot Walkthrough, Objective Fulfilment
10. Conclusion and Future Scope — Conclusion, Limitations, Future Enhancements
11. References
12. Appendices — Sample Data, Key Code Listings, Milestone Outcome Summary

List of Figures

- Fig 3.1 Use-Case Diagram · Fig 4.1 High-Level System Architecture · Fig 4.2 Subsystem Decomposition
- Fig 4.3 Per-Turn Processing Pipeline · Fig 4.4 Hub-and-Spoke Structure · Fig 4.5 DFD Level 0 · Fig 4.6 DFD Level 1
- Fig 4.7 Entity-Relationship Diagram · Fig 6.1–6.5 Use-Case Sequence Diagrams · Fig 9.1–9.5 Application Screenshots

List of Tables

- Table 3.1 Functional Requirements · Table 3.2 Non-Functional Requirements · Table 5.1 Milestone Outcomes
- Table 8.1 Test Cases · Table 9.1 NLU Metrics · Table 9.2 Objective Fulfilment

List of Abbreviations

AI — Artificial Intelligence; API — Application Programming Interface; ARPU — Average Revenue Per User; CRM — Customer Relationship Management; CSAT — Customer Satisfaction; DFD — Data-Flow Diagram; DL — Deep Learning; ER — Entity-Relationship; KPI — Key Performance Indicator; LLM — Large Language Model; ML — Machine Learning; MLP — Multi-Layer Perceptron; NLG — Natural Language Generation; NLP — Natural Language Processing; NLU — Natural Language Understanding; PoC — Proof of Concept; RL — Reinforcement Learning; SLA — Service-Level Agreement; TF-IDF — Term Frequency–Inverse Document Frequency; UAT — User Acceptance Testing; UC — Use Case; UI — User Interface.

Chapter 1 — Introduction

1.1 Overview

The last decade has witnessed a decisive shift in the way organizations interact with their customers. Where once a telephone call to a human agent was the default channel for resolving an account query, customers now expect instant, around-the-clock, self-service resolution through digital text interfaces. This expectation has been amplified by the maturation of conversational AI: chatbots have progressed from brittle, keyword-matching scripts to systems capable of understanding intent, tracking context and generating fluent, human-like replies. This project situates itself at this frontier, focusing on the telecommunications (telecom) sector, where the sheer volume and repetitiveness of support requests make automation both commercially compelling and technically tractable.

The chatbot developed in this project is a text-based conversational agent that automates two of the highest-volume support intents in telecom: resolving disputes about charges on a customer's bill, and recommending personalized promotional offers. Rather than a monolithic script, the system is architected as a small society of cooperating AI agents governed by a central coordinator — a design pattern known as hub-and-spoke. The agent understands free-text input using locally trained machine-learning models, decides on an action using deterministic business logic, and communicates the outcome in natural language, optionally refined by a locally hosted Large Language Model. The entire system runs on a single laptop against synthetic data, making it fully reproducible and free of any privacy concerns associated with real customer records.

1.2 Background of the Domain

Telecom operators manage millions of subscriber accounts, each generating a monthly bill composed of a base plan charge, taxes, and a variable set of usage charges such as international roaming, data overage and premium add-ons. Because these variable charges appear intermittently and are frequently misunderstood, they are the single largest driver of inbound support contacts. A customer who travels abroad and returns to a bill inflated by a roaming charge, or one who exceeds a data cap and incurs an overage fee, will very often contact support in a state of confusion or frustration. Simultaneously, operators run continuous retention and up-sell programmes: loyalty discounts, data add-ons, family-plan consolidations and contract renewals, each targeted at particular customer segments. Surfacing the right offer to the right customer at the right moment is a recognized lever for reducing churn and increasing Average Revenue Per User (ARPU).

Historically, both of these workloads have been serviced by human contact-centre agents following Standard Operating Procedures (SOPs). This model suffers from well-documented weaknesses: high operating cost, inconsistent adherence to policy, long hold times that erode satisfaction, and difficulty scaling to demand spikes. The rule-driven, data-grounded nature of billing explanations and offer eligibility makes them ideal candidates for AI automation, provided the automation can understand messy natural-language input and act on the customer's real account data without fabricating information.

1.3 Problem Statement

The problem addressed by this project can be stated as follows: *design and implement an intelligent conversational chatbot that, given a customer's free-text query, can accurately determine the customer's intent and emotional state, act on the customer's real account to explain or resolve a billing dispute or to recommend and apply a suitable offer, escalate difficult negotiations to a specialist agent, and do all of this in real time while never fabricating account facts and while continuously improving from customer feedback.*

This statement decomposes into several concrete technical challenges: robust intent classification from short, colloquial utterances; sentiment tracking to drive empathetic and escalation behaviour; grounded natural-language generation that stays faithful to account data; a decision engine that encodes billing and offer policy deterministically; a mechanism for handing control between multiple agents; a learning loop that adapts recommendations to observed acceptance; and a usable interface that first verifies the customer's identity.

1.4 Motivation

Three motivations underpin this work. First, **academic**: the project is an opportunity to integrate a broad cross-section of the computer-science curriculum — machine learning, deep learning, natural language processing, reinforcement learning, software architecture and human-computer interaction — into a single, coherent, working system. Second, **practical**: billing disputes and offer recommendation are real, high-value problems whose automation delivers measurable business impact through reduced handling cost and improved customer satisfaction. Third, **methodological**: the project deliberately favours explainable, locally trained models and deterministic business logic over an opaque end-to-end Large Language Model, demonstrating that a hybrid design can achieve fluent, human-like interaction while retaining the transparency, controllability and factual reliability that regulated domains such as billing demand.

1.4a The Economics of Billing Contacts and Churn

A brief consideration of the economics clarifies why the two chosen intents are worth automating well. In telecom, the cost of acquiring a new subscriber is substantial, typically far exceeding the cost of retaining an existing one, so any interaction that reduces the risk of a customer leaving has outsized value. Billing confusion is a recognized driver of dissatisfaction and, ultimately, of churn: a customer who receives an unexpectedly high bill and then endures a long, unhelpful support experience is materially more likely to defect to a competitor. Conversely, a support interaction that resolves the confusion quickly, treats the customer fairly, and — where appropriate — proactively offers a well-matched discount can convert a moment of frustration into a moment of loyalty. The offer side of the system is the natural complement: surfacing a relevant retention offer at the point of contact, rather than through an untargeted mass campaign, both reduces churn and lifts revenue per user. Automating these interactions with an agent that is instantly available, consistent and personalized therefore attacks cost and churn simultaneously, which is the commercial rationale underpinning the technical work described in this report.

1.5 Objectives

The project is guided by five objectives, which also structure its evaluation:

1. To design and develop an intelligent conversational AI chatbot capable of delivering automated, accurate and real-time customer support using advanced NLP techniques.
2. To enhance chatbot response quality and efficiency through machine-learning optimization aimed at improving customer satisfaction and reducing average response time.
3. To implement deep-learning-based architectures for Natural Language Understanding and response generation, enabling context-aware and human-like interactions.
4. To integrate reinforcement-learning strategies for continuous chatbot improvement based on user interactions and feedback loops.
5. To evaluate chatbot performance using key metrics such as accuracy, intent-classification precision, response relevance, operational efficiency and user-satisfaction scores.

1.6 Scope of the Project

The scope is deliberately bounded to keep the proof-of-concept focused and demonstrable. **In scope**: text-based conversation; identity validation; billing-dispute resolution for roaming and overage charges; explanation of bill composition; provisional credits, goodwill waivers and dispute-ticket creation; personalized offer recommendation and application; a hub-and-spoke handoff to a retention specialist with a negotiation ladder; a reinforcement-learning feedback loop for offers; live signal and KPI monitoring; and a Streamlit user interface. **Out of scope** for this phase: voice/telephony channels; integration with live production CRM or billing back-ends (synthetic adapters are used instead); transfer to a human agent (both agents are AI); a formal supervisor email-approval workflow (approval is simulated within the interface); and multilingual support. These exclusions are discussed further as future scope in Chapter 10.

1.7 Proof-of-Concept Nature and Assumptions

This project is explicitly a proof of concept (PoC). Its purpose is to validate the architecture, the interaction design and the integration of the various AI components end-to-end, rather than to deliver a production-hardened service. Two assumptions follow from this. First, all data is **synthetic**: three representative customers with fabricated bills, usage histories and interaction logs stand in for a real subscriber base, eliminating privacy concerns and making the system fully reproducible. Second, the machine-learning models are trained on a compact, hand-labelled seed corpus; this keeps the models transparent and fast to train on commodity hardware, at the cost of held-out accuracy that would rise substantially with a larger, log-mined corpus. These assumptions are stated openly so that the evaluation in Chapter 9 can be interpreted correctly.

1.8 Organization of the Report

The remainder of this report is organized as follows. **Chapter 2** surveys the relevant literature and background across conversational AI, NLP, NLU, sentiment analysis, deep learning, Large Language Models, reinforcement learning, recommender systems and dialogue management, and identifies the research gap. **Chapter 3** presents the system analysis: the limitations of existing systems, the proposed system, a feasibility study, and functional and non-functional requirements. **Chapter 4** details the system design, including the high-level architecture, subsystem decomposition, the per-turn processing pipeline, the hub-and-spoke design, data-flow diagrams and the data model. **Chapter 5** describes the methodology milestone by milestone and records the outcome of each. **Chapter 6** explains every use case in detail with an accompanying sequence diagram. **Chapter 7** covers the implementation, including a walkthrough of the codebase folder structure and the technology stack. **Chapter 8** presents the testing approach and results. **Chapter 9** reports the evaluation results and maps them to the objectives. **Chapter 10** concludes and outlines future scope. References and appendices follow.

Chapter 2 — Literature Review and Background

2.1 Evolution of Conversational AI and Chatbots

The idea of a machine conversing in natural language is as old as the field of artificial intelligence itself. The first widely known chatbot, ELIZA, was created by Joseph Weizenbaum in 1966 and simulated a Rogerian psychotherapist through simple pattern-matching and canned transformations of the user's input. Despite its simplicity, ELIZA demonstrated that users would readily attribute understanding to a system that merely reflected their words back at them — an observation now called the ELIZA effect. PARRY (1972) followed, modelling a paranoid patient with a richer internal state. These early systems were entirely rule-based: they matched keywords and applied scripted responses, with no genuine understanding of meaning.

The subsequent decades saw incremental progress through better pattern languages such as AIML (Artificial Intelligence Markup Language), which powered award-winning bots like A.L.I.C.E., and through the rise of statistical NLP. The commercial inflection point arrived with virtual assistants such as Apple's Siri, Amazon's Alexa and Google Assistant, which combined speech recognition, statistical intent classification and structured back-end actions. In parallel, task-oriented dialogue systems matured in research, formalizing the pipeline of NLU, dialogue state tracking, dialogue policy and NLG. Most recently, the advent of large pre-trained transformer language models has enabled open-domain conversation of unprecedented fluency. The system in this project draws on this lineage selectively: it adopts the disciplined task-oriented pipeline for reliability, while borrowing the fluency of Large Language Models for the final phrasing step only.

2.2 Natural Language Processing Fundamentals

Natural Language Processing is the sub-field of AI concerned with enabling computers to process and analyse human language. A typical NLP pipeline begins with **text pre-processing**: tokenization (splitting text into words or sub-words), normalization (lower-casing, expanding contractions, removing punctuation), and sometimes stemming or lemmatization (reducing words to a common root). The cleaned text must then be converted into a numerical representation that a model can consume. Classical approaches include the **bag-of-words** model, which represents a document as a vector of word counts, and **TF-IDF** (Term Frequency–Inverse Document Frequency), which weights each term by how frequently it appears in a document relative to how common it is across the whole corpus, thereby emphasizing distinctive words. Modern approaches instead learn dense **word embeddings** (Word2Vec, GloVe, fastText) or contextual embeddings from transformer models (BERT and its successors) that capture semantic similarity. This project uses TF-IDF because it is transparent, fast, requires no pre-trained embeddings, and performs strongly on the short, domain-specific utterances characteristic of support chat.

2.3 Natural Language Understanding

Natural Language Understanding is the component of a conversational system responsible for converting a raw utterance into a structured, machine-actionable representation. Two tasks are central. **Intent classification** assigns the utterance to one of a fixed set of intents — for example, distinguishing “why is my bill higher this month” (a bill-explanation intent) from “I never went to Mexico” (a factual-dispute intent). This is a supervised multi-class text classification problem, commonly solved with algorithms ranging from logistic regression and support-vector machines over TF-IDF features to fine-tuned transformers. **Entity extraction** (or slot filling) identifies the structured arguments within the utterance — a charge identifier, a monetary amount, a date range, or a yes/no confirmation. Entity extraction is frequently handled with a combination of rules/regular expressions and sequence-labelling models such as conditional random fields or BiLSTM-CRF networks. The NLU stack in this project combines a neural intent classifier and sentiment analyser with rule-based entity extraction, a pragmatic and explainable configuration for a bounded domain.

2.4 Sentiment and Emotion Analysis

Sentiment analysis determines the affective polarity of text — typically positive, negative or neutral — and is a cornerstone of customer-experience analytics. In a support setting it serves two operational purposes: it lets the agent modulate tone

(becoming more empathetic as frustration rises) and it acts as a trigger for escalation. Techniques range from **lexicon-based** methods, which sum the polarities of words found in a sentiment dictionary such as VADER or SentiWordNet, to supervised classifiers trained on labelled corpora, to transformer-based models fine-tuned for sentiment. Emotion analysis extends this to finer categories such as anger, frustration, satisfaction and disappointment. This project trains a compact neural sentiment classifier that outputs a three-class label together with a continuous signed score, allowing the policy engine to reason about not just the current sentiment but its trajectory across a conversation.

2.5 Machine Learning for Text Classification

Supervised machine learning frames intent and sentiment detection as learning a function from labelled examples. The classical workflow — feature extraction (e.g. TF-IDF), followed by a discriminative classifier — remains highly effective for small, well-scoped datasets. Logistic regression and linear support-vector machines offer strong baselines with excellent interpretability. Naïve Bayes, despite its independence assumptions, is a robust and fast text classifier. Ensemble methods such as random forests and gradient boosting can capture non-linear feature interactions. The key practical concerns are class imbalance, overfitting on small corpora (mitigated by regularization and cross-validation), and the need for a sensible fallback when the model is uncertain. This project adopts a neural classifier (described next) with an L2-regularized objective and a deterministic keyword fallback for low-confidence predictions.

2.6 Deep Learning and Neural Networks

Deep learning uses multi-layer neural networks to learn hierarchical representations directly from data. The **Multi-Layer Perceptron (MLP)** is the foundational feed-forward architecture: an input layer, one or more hidden layers of neurons applying non-linear activation functions, and an output layer. Trained by back-propagation and gradient descent, an MLP can approximate complex non-linear decision boundaries, and even over TF-IDF features it can capture interactions that a purely linear model cannot. For sequential text, **recurrent neural networks** (RNNs) and their gated variants (LSTM, GRU) model word order, while **convolutional neural networks** capture local n-gram patterns. The most significant recent development is the **transformer** architecture, whose self-attention mechanism captures long-range dependencies and underpins virtually all state-of-the-art language models. In this project the intent and sentiment classifiers are implemented as MLP neural networks; this satisfies the objective of deep-learning-based NLU while remaining lightweight enough to train in seconds on a CPU, a deliberate trade-off appropriate to a reproducible proof of concept.

2.7 Large Language Models and Natural Language Generation

Natural Language Generation is the task of producing coherent natural-language text from an internal representation. Traditional NLG followed a pipeline of content determination, sentence planning and surface realization, often using templates. The advent of **Large Language Models** — transformer networks with billions of parameters pre-trained on vast text corpora — has transformed NLG: models such as the GPT family, Claude, LLaMA and their derivatives can generate fluent, contextually appropriate text from a natural-language prompt. However, LLMs are prone to **hallucination**: confidently stating plausible but false information. In a billing context this is unacceptable — an agent must never invent a charge or an offer amount. This project therefore adopts a **grounded, hybrid NLG** strategy: deterministic templates carry the authoritative facts, and a locally hosted LLM (LLaMA 3.2, served through Ollama) is invoked only to rephrase those templates in a warm, human voice, under an explicit instruction not to introduce any new facts. This preserves fluency while eliminating the hallucination risk, and the LLM is entirely optional — the system degrades gracefully to pure templates if it is unavailable.

2.8 Reinforcement Learning and Multi-Armed Bandits

Reinforcement Learning (RL) is a paradigm in which an agent learns to make decisions by interacting with an environment and receiving rewards. Unlike supervised learning, RL has no labelled examples; instead, the agent must balance **exploration** (trying actions to gather information) against **exploitation** (choosing the action currently believed best). The **multi-armed bandit** is the simplest RL setting: an agent repeatedly chooses among several actions (“arms”), each returning a stochastic

reward, and seeks to maximize cumulative reward. The **epsilon-greedy** strategy is a classic, easily interpretable solution: with probability epsilon the agent explores a random arm, and otherwise it exploits the arm with the highest estimated value. Contextual bandits extend this by conditioning the choice on features of the current situation. In this project, offer recommendation is framed as a bandit problem: each offer is an arm, a customer accepting an offer yields a reward of one and a decline yields zero, and the learned acceptance value is blended into the deterministic recommendation score so that offers customers actually accept rise in the ranking over time. This realizes the objective of continuous improvement from feedback loops.

2.9 Recommender Systems

Recommender systems predict the items a user is most likely to value. The two classical families are **collaborative filtering**, which recommends items favoured by similar users, and **content-based filtering**, which recommends items similar to those a user has liked, using item and user features. Hybrid and knowledge-based recommenders combine these with explicit business rules. In a telecom offer-recommendation context, pure collaborative filtering is often impractical because eligibility is governed by hard business rules (tier, tenure, plan type) rather than taste. This project therefore employs a **knowledge-based, rule-plus-scoring recommender**: customer attributes are bucketed into low/medium/high bands, offers are filtered by eligibility rules, and a weighted heuristic produces a score. This deterministic core is then refined by the reinforcement-learning bandit of Section 2.8, giving a recommender that is both explainable (every recommendation has a stated reason) and adaptive.

2.10 Dialogue Management and Multi-Agent Systems

Dialogue management governs the flow of a conversation: it tracks the dialogue state, decides the next system action, and maintains context across turns. Task-oriented systems often model each task as a finite-state machine or a frame-filling process. As conversational applications grow, a single monolithic policy becomes unwieldy, motivating **multi-agent** designs in which specialized agents handle different sub-tasks. The **hub-and-spoke** (or orchestrator) pattern is a widely used multi-agent topology: a central coordinator (the hub) receives every input, decides which specialized agent (spoke) should handle it, and mediates all communication, so that the spokes remain decoupled and never invoke one another directly. This yields clean separation of concerns, controlled hand-offs, and easy extensibility. The present system implements exactly this pattern, with a coordinator routing turns between a billing-and-offers spoke and a retention-specialist spoke, and it augments the flows with a lightweight policy engine that converts detected signals into tone and escalation directives.

2.11 Related Work — AI in Telecom Customer Support

The application of AI to telecom customer support is an active area in industry. Operators have deployed intent-routing bots that triage incoming queries, frequently-asked-question retrieval systems, and increasingly, generative assistants layered over knowledge bases through retrieval-augmented generation. Commercial contact-centre platforms provide sentiment monitoring, agent-assist suggestions and automated after-call summaries. Academic and industrial work has also explored churn prediction, next-best-action recommendation and dispute-resolution automation. The distinctive contribution of this project relative to that landscape is its integration: rather than addressing a single sub-problem, it combines identity validation, deep-learning NLU, grounded NLG, a knowledge-based recommender, reinforcement-learning optimization and a multi-agent hub-and-spoke dialogue manager into one coherent, runnable, observable system, while remaining fully local, reproducible and privacy-safe.

2.11a Comparison of Design Approaches

It is instructive to contrast the three broad design approaches available for a support chatbot, because the choices made in this project are a direct response to their respective trade-offs. The first approach is the **purely rule-based** bot, built from hand-crafted patterns and decision trees. Its strengths are total predictability, transparency and zero dependence on training data; its weaknesses are brittleness in the face of unexpected phrasing, an inability to gauge sentiment, laborious maintenance as

rules proliferate, and a stilted conversational feel. The second approach is the **fully generative** agent, in which a single large language model both understands and responds. Its strengths are remarkable fluency and flexibility; its weaknesses are the risk of hallucinating account facts, the difficulty of constraining it to policy, its opacity, its latency and its dependence on a powerful (often remote) model. The third approach — the one adopted here — is a **hybrid, modular** design that uses learned models for understanding, deterministic logic for decisions, and a language model only for phrasing, under strict grounding.

The hybrid approach inherits the reliability and transparency of the rule-based design in the consequential parts of the system — intent handling, eligibility, credit and ticket decisions — while borrowing the fluency of the generative approach in the one place where fluency matters and errors are harmless: the surface phrasing of a reply whose facts have already been fixed. It also adds a dimension neither of the pure approaches offers cheaply: a learning loop that adapts recommendations to observed behaviour without retraining a large model. The cost of the hybrid approach is greater architectural complexity — more moving parts, more interfaces — but this cost is precisely what the disciplined subsystem decomposition of Chapter 4 is designed to manage. This comparison explains why the project does not simply wrap a large language model in a chat window: doing so would sacrifice the correctness and auditability that a billing domain cannot compromise.

2.12 Research Gap and Positioning

Two gaps motivate the design choices of this project. First, purely LLM-driven support agents, while fluent, are difficult to constrain and can hallucinate account facts — an unacceptable risk in billing. Second, purely rule-based bots are reliable but rigid and unnatural. The gap, therefore, is a design that captures the fluency of generative models and the reliability of deterministic logic simultaneously, while remaining transparent and adaptive. This project addresses that gap through its hybrid, grounded, hub-and-spoke architecture: deterministic understanding and decision-making for correctness, an optional LLM for fluency, and a reinforcement-learning loop for adaptivity. The following chapters develop this design in full.

2.13 Evaluation Metrics for Conversational Systems

Evaluating a conversational system requires metrics at several levels. At the **component** level, classification tasks such as intent detection and sentiment analysis are measured with **accuracy** (the proportion of correct predictions), **precision** (the proportion of predicted positives that are correct), **recall** (the proportion of actual positives that are recovered) and the **F1-score** (the harmonic mean of precision and recall, which balances the two and is especially informative under class imbalance). A **confusion matrix** exposes which classes are mutually confused. Because a single train/test split can be misleading on small corpora, **stratified splitting** and **cross-validation** are used to obtain stable estimates. At the **conversation** level, task-oriented systems are judged by **task-completion rate** (here, the resolution rate), the **number of turns to completion**, and **latency**. At the **experience** level, **user-satisfaction** is captured directly (for example through thumbs feedback yielding a Customer Satisfaction, or CSAT, proxy) or inferred from sentiment trajectories. This project reports metrics at all three levels: component accuracy/precision/recall/F1, conversation resolution rate and latency, and a satisfaction proxy, in direct correspondence with the fifth project objective.

2.14 Responsible, Explainable and Trustworthy AI

Deploying AI in a customer-facing, financially sensitive domain such as billing raises questions of responsibility and trust that a purely technical design cannot ignore. Three properties are particularly relevant. **Explainability** is the degree to which a system's decisions can be understood by a human; a recommendation that states its reason (“a loyalty reward for your long tenure”) is far more trustworthy than an opaque score. **Factual reliability** is the guarantee that the system does not assert falsehoods; in billing this is paramount, since a fabricated charge or credit could have legal and financial consequences. **Privacy** demands that personal data be protected and disclosed only to verified parties. The design decisions in this project are shaped by these principles: deterministic, reason-bearing recommendations provide explainability; the grounded-NLG discipline, in which the language model may rephrase but never invent, provides factual reliability; and the mandatory identity-validation step, together with the exclusive use of synthetic data, addresses privacy. In this sense the system is not merely functional but deliberately trustworthy by construction.

2.14a Ethical Considerations and Data Privacy

Automating customer support in a financial domain carries ethical responsibilities that extend beyond correctness. A support agent influences whether a customer is charged, credited or offered a discount, so fairness and transparency are essential: comparable customers should receive comparable treatment, and the basis of any decision should be explainable on request. Automated systems can also inadvertently encode bias if trained on skewed historical data, which argues for transparent, rule-bearing decision logic in the sensitive billing path and for careful curation of any learned components. Data privacy is equally central: customer accounts contain personal and financial information that must be protected, disclosed only after authentication, retained only as long as necessary, and handled in accordance with data-protection regulation. This project embodies these concerns by construction — it gates all account access behind identity validation, uses only synthetic data so that no real individual is ever exposed, keeps the consequential billing decisions deterministic and explainable, and confines the generative model to non-consequential phrasing. While a production deployment would require formal privacy impact assessment, consent management and audit logging, the proof of concept is designed to be extended in that direction rather than to work against it.

2.15 Software Engineering Practices for AI Systems

Building a reliable AI application is as much a software-engineering challenge as a modelling one. Good practice separates concerns into cohesive modules with narrow interfaces, so that the model layer, the business logic and the presentation can evolve independently. Configuration is externalized (here, into an environment file) so that behaviour can change without code edits. Deterministic components are favoured wherever correctness matters, with learned or generative components confined to where their flexibility adds value and their errors are tolerable. Every external dependency is given a fallback so that the system degrades gracefully rather than failing outright. Finally, observability is engineered in from the start: structured logging of each turn's signals and outcomes turns the running system into a source of evaluation data. These practices, drawn from the emerging discipline sometimes called MLOps, are applied throughout this project and are visible in its modular folder structure, its configuration-driven provider switch, its graceful fallbacks and its session logging.

Chapter 3 — System Analysis

3.1 The Existing System and Its Limitations

In the conventional telecom support model, a customer with a billing query or an interest in offers contacts a human agent by phone or through a live-chat window. The agent authenticates the customer, looks up the account in a Customer Relationship Management (CRM) and billing system, interprets the customer's problem, consults Standard Operating Procedures to decide on a resolution, and records the outcome. Where the situation exceeds the agent's authority — for instance, a large goodwill credit or a difficult retention negotiation — the case is escalated to a supervisor. While functional, this model exhibits several limitations:

- **High operating cost:** human agents are expensive and support volumes are large and bursty.
- **Long wait times:** queueing during peak periods directly degrades customer satisfaction.
- **Inconsistency:** different agents apply policy differently, producing uneven outcomes.
- **Limited availability:** staffed support is rarely truly 24×7 without significant cost.
- **Reactive offers:** relevant promotions are seldom surfaced proactively at the point of contact.
- **Poor observability:** the emotional trajectory and intent of a conversation are rarely captured as structured data for analysis.

Some operators have layered simple menu-driven or keyword bots on top of this model, but these are notoriously rigid: they fail when the customer phrases a request in an unexpected way, cannot track sentiment, and offer no learning mechanism. There is a clear need for a system that understands natural language robustly, acts on real account data, personalizes offers, escalates gracefully, and improves over time.

3.2 The Proposed System

The proposed system is a text-based conversational AI chatbot that addresses these limitations directly. After validating the customer's identity, it accepts free-text queries and, for each turn, understands the intent and sentiment, extracts relevant entities, decides on an action against the customer's synthetic account, and replies in grounded natural language. It resolves billing disputes by explaining charges and offering evidence-backed remedies (a provisional credit, a goodwill waiver, or a formal dispute ticket), and it recommends personalized offers scored against the customer's profile and refined by a learning loop. When a negotiation becomes difficult, a coordinator hands the conversation to a specialist retention agent that follows an escalating offer ladder. Throughout, the system exposes live signals and KPIs for monitoring. The key advantages over the existing model are instant availability, consistent policy application, proactive and adaptive personalization, structured observability, and zero marginal cost per conversation.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The system is technically feasible with widely available, open-source tools. The NLU models are trained with scikit-learn and run on a CPU in seconds; the user interface is built with Streamlit; the optional Large Language Model is served locally by Ollama; and all data is stored as JSON files, avoiding the need to provision a database. The entire stack installs from a single requirements file and launches with one command, confirming that no specialized hardware or proprietary software is required.

3.3.2 Economic Feasibility

Because the project relies exclusively on free and open-source software and runs on commodity hardware, its development cost is effectively limited to time. In a production setting, the economic case rests on deflecting a proportion of human-handled contacts to the automated agent, whose marginal cost per conversation is negligible compared with a staffed interaction.

3.3.3 Operational Feasibility

Operationally, the system is designed for ease of use: customers interact through a familiar chat interface requiring no training, and the conversation is guided. For operators, the modular architecture allows the synthetic data adapters to be replaced with live CRM/billing connectors without altering the dialogue logic. The grounded NLG and deterministic decision core make the system's behaviour auditable, which is essential for operational acceptance in a regulated billing context.

3.4 Requirement Analysis

3.4.1 Functional Requirements

ID	Requirement
FR-1	The system shall authenticate a customer via email and password and then verify identity via the last four digits of the ID on file before serving any account data.
FR-2	The system shall classify each customer utterance into one of the defined intents and detect its sentiment.
FR-3	The system shall extract relevant entities (charge references, amounts, dispute topic, yes/no confirmations) from utterances.
FR-4	The system shall explain the customer's current bill and surface any unusual charge (roaming or overage).
FR-5	The system shall present evidence for a disputed charge and offer resolution options appropriate to the customer's tier.
FR-6	The system shall apply a provisional credit or goodwill waiver, or create a persisted dispute ticket, according to the customer's choice.
FR-7	The system shall compute personalized offer recommendations and pitch the highest-ranked eligible offer.
FR-8	The system shall apply an accepted offer and record the acceptance as reinforcement-learning feedback.
FR-9	The coordinator shall hand off the conversation to the retention specialist upon haggling, sustained negative sentiment, exhausted offers, or an explicit request.
FR-10	The retention specialist shall follow an escalating negotiation ladder and apply accepted offers under a simulated supervisor approval.
FR-11	The system shall answer ticket-status queries.
FR-12	The system shall display live per-turn signals (intent, sentiment, active agent, resolution status) and aggregate KPIs.
FR-13	The system shall accept explicit user feedback (thumbs up/down) on replies.

3.4.2 Non-Functional Requirements

ID	Requirement
NFR-1 (Performance)	The deterministic per-turn processing shall complete within a few milliseconds; LLM-phrased replies within a few seconds.
NFR-2 (Reliability)	The system shall degrade gracefully to templated responses if the LLM or any optional dependency is unavailable.
NFR-3 (Correctness)	Generated replies shall never contain fabricated account facts; all figures shall derive from the data layer.
NFR-4 (Usability)	The interface shall require no training and shall guide the customer through login, verification and chat.
NFR-5 (Security/Privacy)	Account data shall be served only after identity verification; the system shall use synthetic data exclusively.
NFR-6 (Maintainability)	Business logic shall be modular so data adapters and the LLM provider can be swapped via configuration.
NFR-7 (Observability)	Every turn shall emit structured signals; sessions shall be logged for KPI aggregation.
NFR-8 (Portability)	The system shall run on commodity hardware (macOS/Linux) with a single-command launch.

3.5 Use-Case Identification

Analysis of the requirements yields the following primary use cases, each elaborated with a sequence diagram in Chapter 6: UC-00 Identity Validation; UC-01 Billing Dispute Resolution; UC-02 Offer Recommendation; UC-03 Retention Handoff and Negotiation; and UC-04 Ticket Status Query. The single external actor is the Customer; internally, the AI agents Aria (billing/offers) and Ray (retention), the recommendation engine and the reinforcement-learning component participate.

3.6 Use-Case Diagram

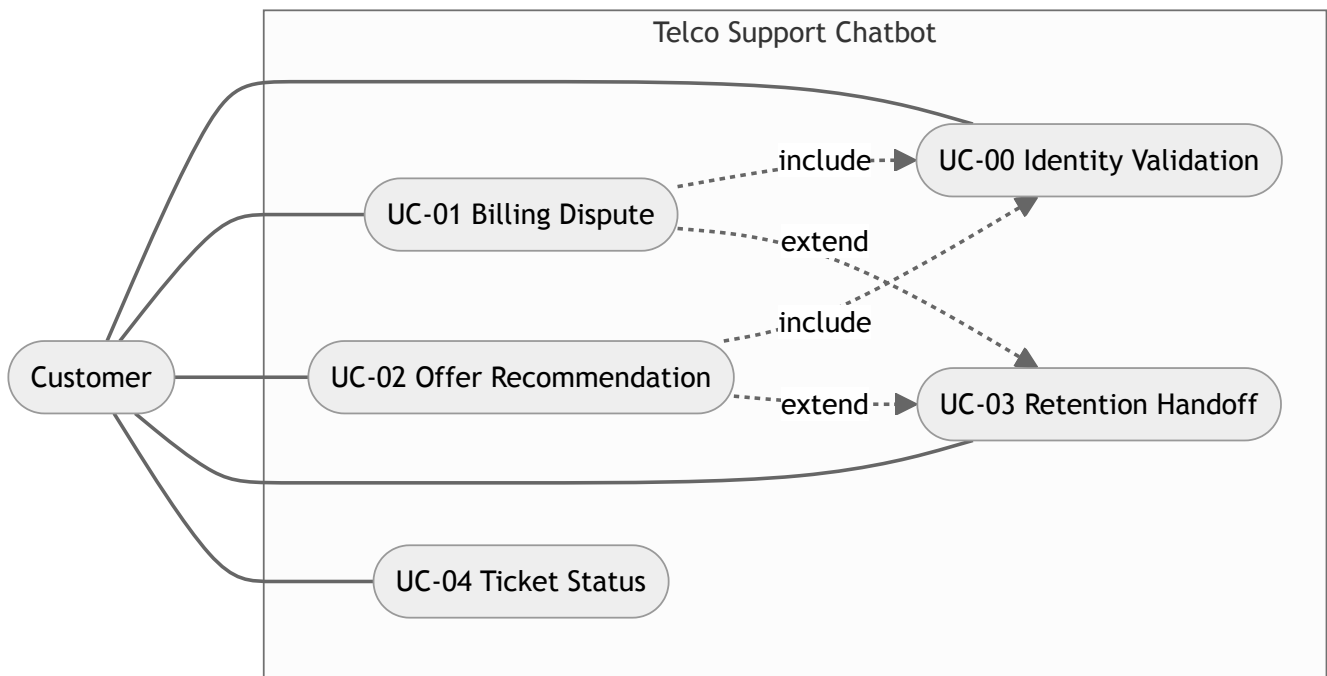


Fig 3.1 — Use-Case Diagram. Billing and offer use cases include identity validation and may extend into a retention handoff.

3.7 Hardware and Software Requirements

Category	Requirement
Processor	Any modern multi-core CPU (Apple Silicon / x86-64); no GPU required
Memory	8 GB RAM minimum (16 GB recommended if running the local LLM)
Disk	~2 GB for the environment; ~2 GB additional for the LLM model
Operating System	macOS or Linux (Windows via WSL)
Language / Runtime	Python 3.9+
Core libraries	Streamlit, scikit-learn, NumPy, pandas, joblib, python-dotenv, requests, PyYAML
Optional LLM	Ollama runtime with the llama3.2 model

Chapter 4 — System Design

4.1 Design Goals and Guiding Principles

The design is governed by four principles. **Thick brain, thin interface:** all understanding, decision-making and action live in the Python brain; the Streamlit interface only renders. **Deterministic core, optional intelligence:** facts and decisions are computed by rules and data, while the LLM is an optional enhancement confined to phrasing. **Graceful degradation:** every dependency has a fallback, so the demonstration never hard-fails. **Observability by design:** each turn emits structured signals that the interface and the KPI aggregator consume. These principles reconcile the competing demands of fluency and reliability that Chapter 2 identified as the research gap.

4.2 High-Level Architecture

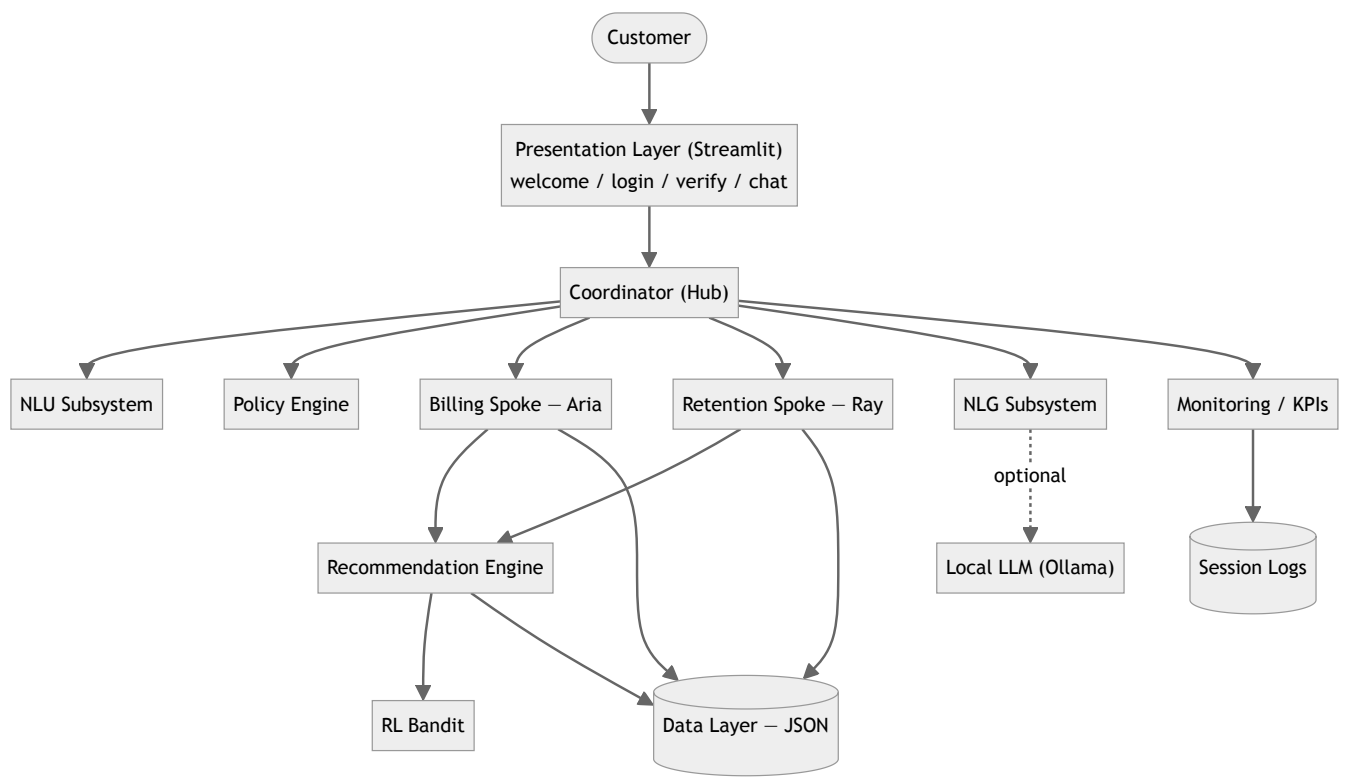
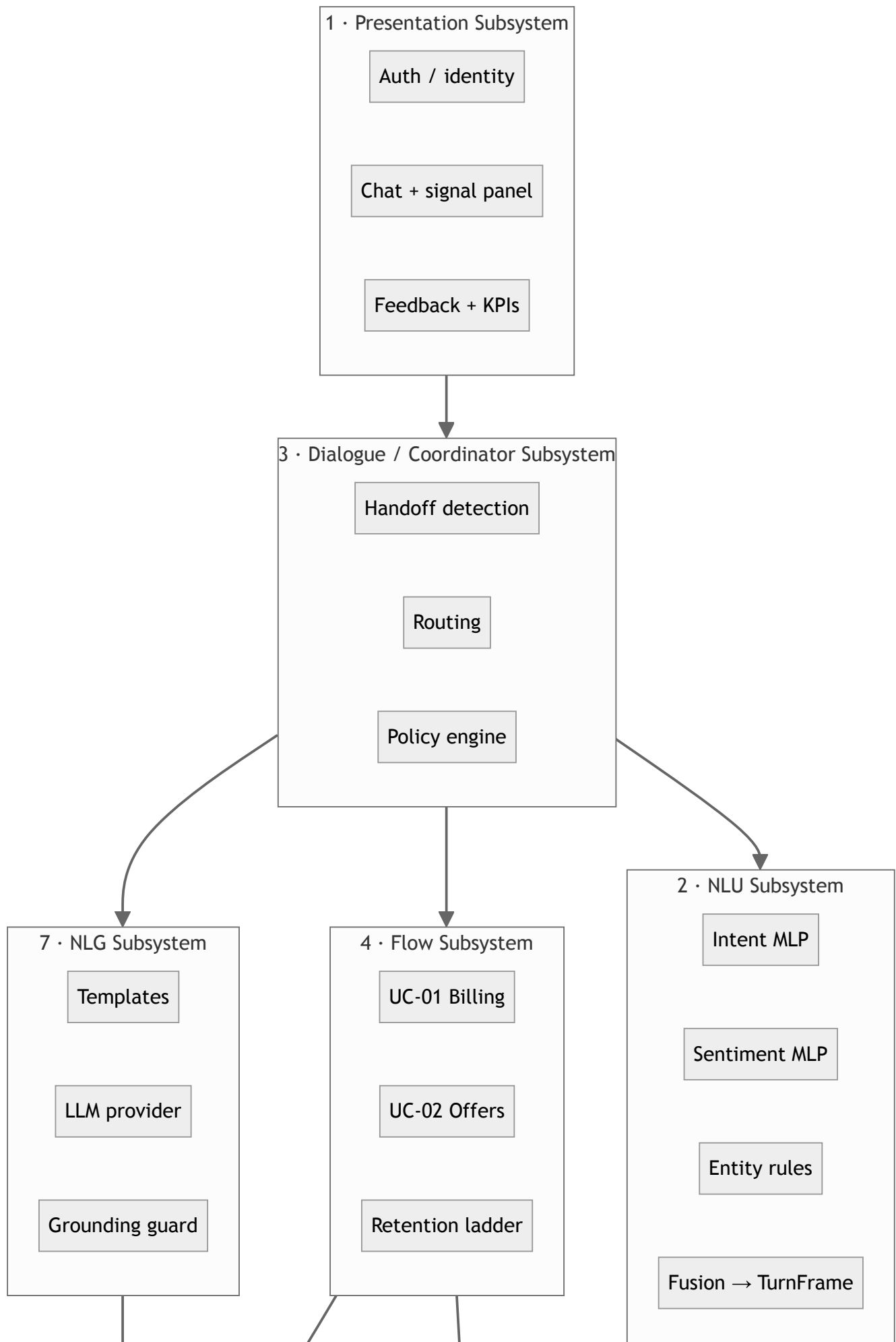


Fig 4.1 — High-Level System Architecture.

The architecture is layered. The **presentation layer** handles authentication and chat rendering. The **coordinator** is the hub that orchestrates each turn. Beneath it sit the specialized subsystems — NLU, policy, the two agent spokes, recommendation, reinforcement learning and NLG — over a shared **data layer**. A cross-cutting **monitoring** concern captures signals and KPIs.

4.3 Subsystem Decomposition

The brain decomposes into eight cohesive subsystems, each with a single responsibility and a narrow interface, shown in Fig 4.2 and detailed in the table that follows.



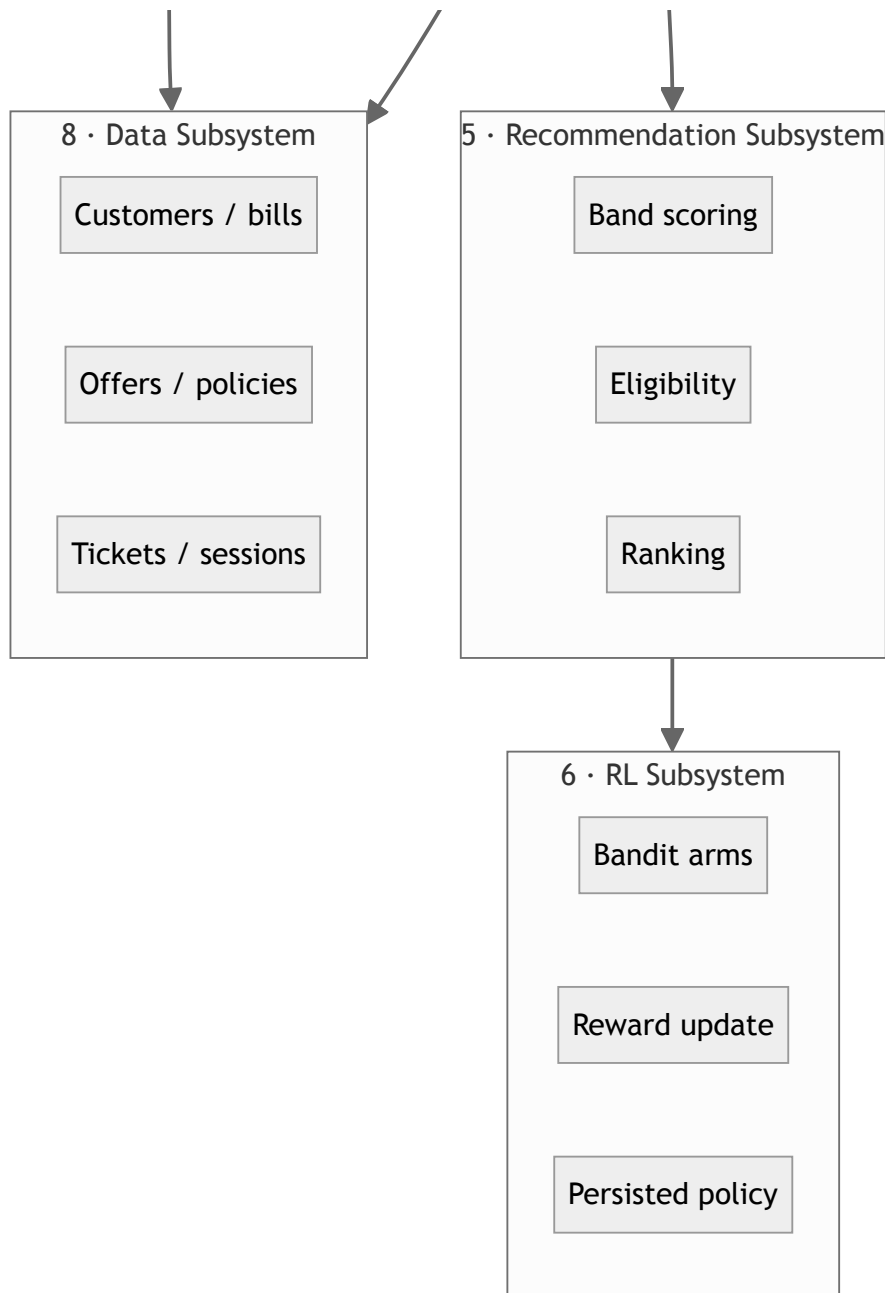
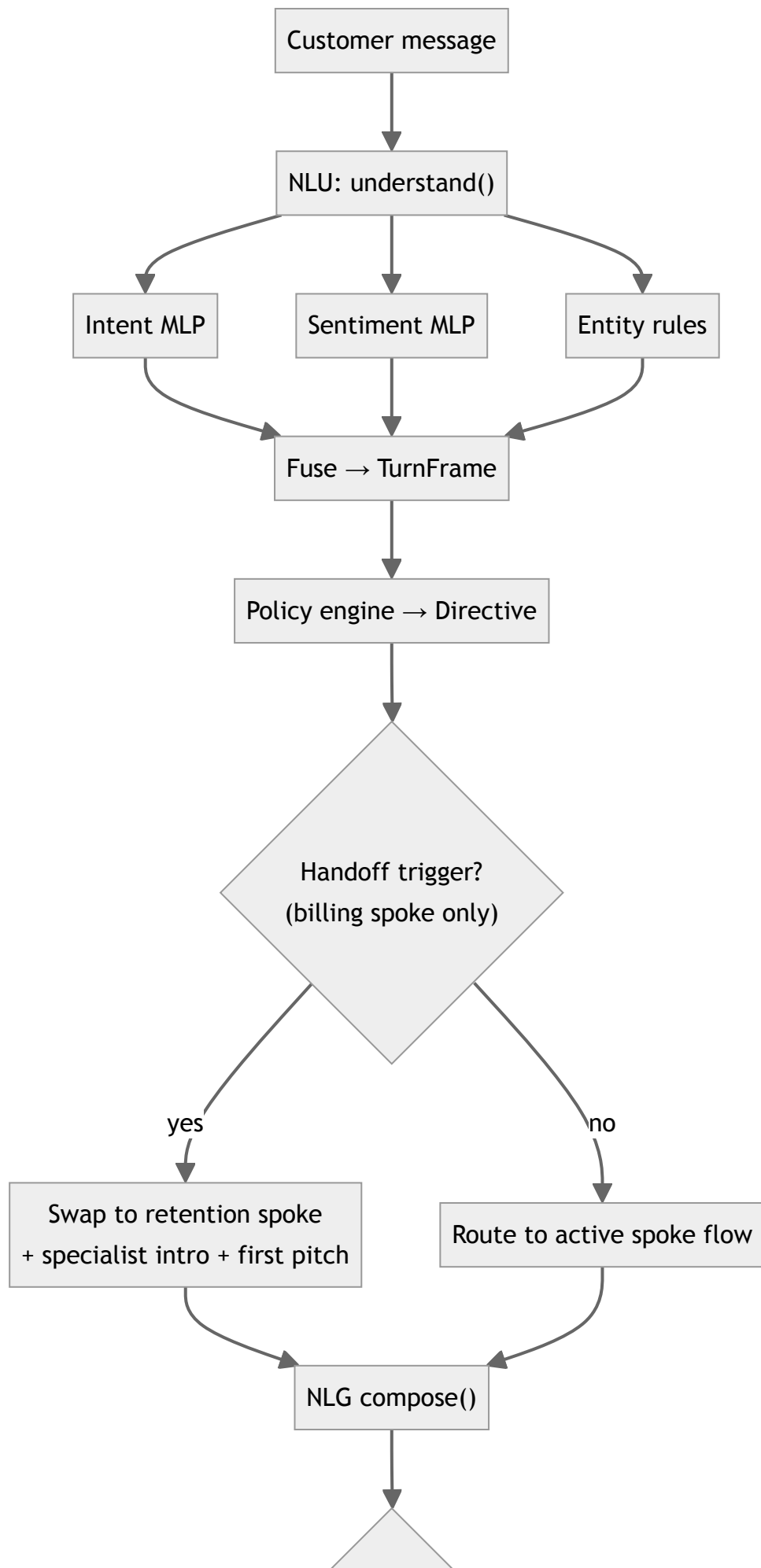


Fig 4.2 — Subsystem Decomposition (eight subsystems).

#	Subsystem	Responsibility	Primary modules
1	Presentation	Authentication, identity check, chat rendering, signal/KPI panels, feedback capture	<code>ui/streamlit_app.py</code> , <code>app/auth.py</code>
2	NLU	Intent + sentiment classification, entity extraction, fusion into a TurnFrame	<code>app/nlu/*</code>
3	Dialogue / Coordinator	Hub routing, handoff detection, policy directives	<code>app/coordinator.py</code> , <code>app/policy_engine.py</code>
4	Flow	Use-case state machines (billing, offers, retention ladder)	<code>app/flows/*</code>
5	Recommendation	Band computation, eligibility filtering, weighted scoring, ranking	<code>app/recommendation_engine.py</code>
6	Reinforcement Learning	Bandit arms, reward updates, persisted policy	<code>app/rl/bandit.py</code>
7	NLG	Grounded templates, pluggable LLM provider, anti-hallucination guard	<code>app/nlg/*</code>
8	Data	Access to customers, bills, offers, policies, tickets and session logs	<code>app/data_store.py</code> , <code>data/*.json</code>

4.4 Per-Turn Processing Pipeline

Every customer message is processed by a fixed pipeline inside the coordinator. This deterministic ordering is the backbone of the system's reliability.



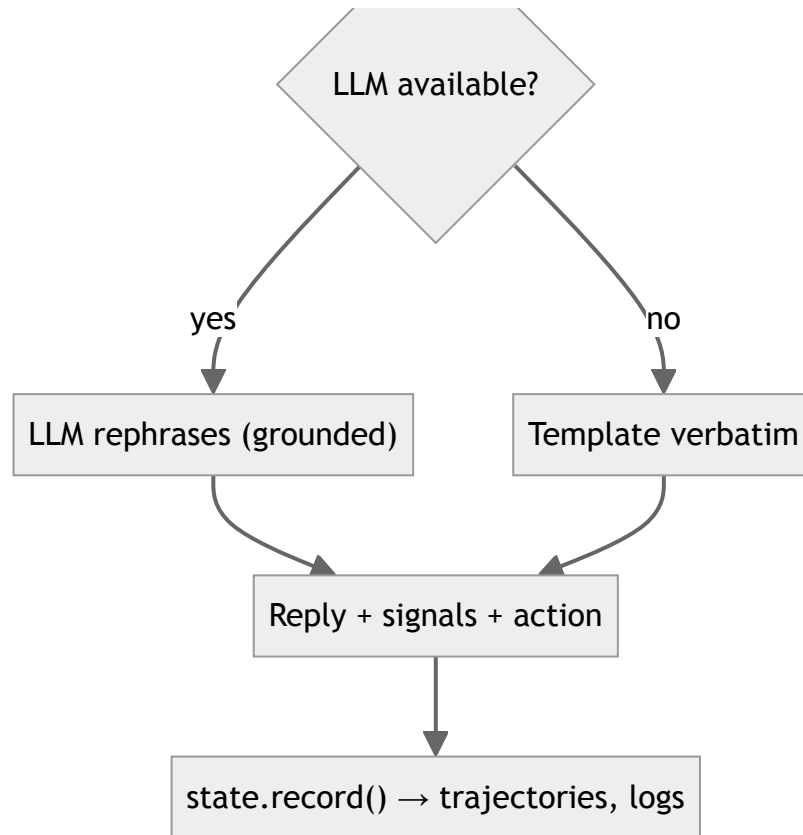


Fig 4.3 — Per-Turn Processing Pipeline.

4.5 Hub-and-Spoke Design

The dialogue subsystem implements the hub-and-spoke pattern. The coordinator is the hub; the billing-and-offers agent (Aria) and the retention specialist (Ray) are the spokes. Critically, the spokes never communicate directly — all control transfer passes through the hub. This decoupling means a new spoke could be added without modifying existing ones, and the hand-off logic is centralized and auditable.

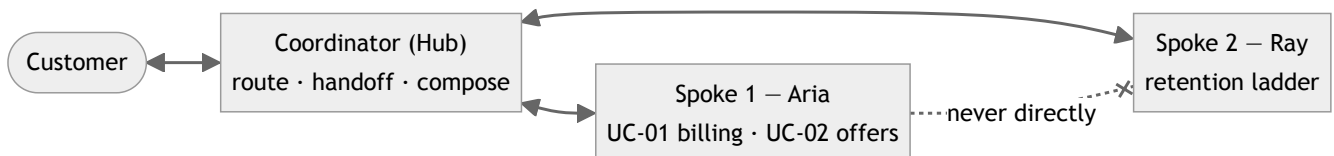


Fig 4.4 — Hub-and-Spoke Structure.

The coordinator hands off from Aria to Ray when any of the following triggers fire: the flow signals it cannot resolve (offers exhausted or a credit exceeding authority), the customer explicitly asks for escalation, the customer haggles, or the policy engine raises the escalation flag due to sustained negative sentiment across at least two turns. The two spokes are also distinguished visually and by persona in the interface so the hand-off is clear to the customer.

4.6 Data-Flow Diagrams

The context-level (Level 0) data-flow diagram treats the whole chatbot as a single process exchanging data with the customer and the data store.

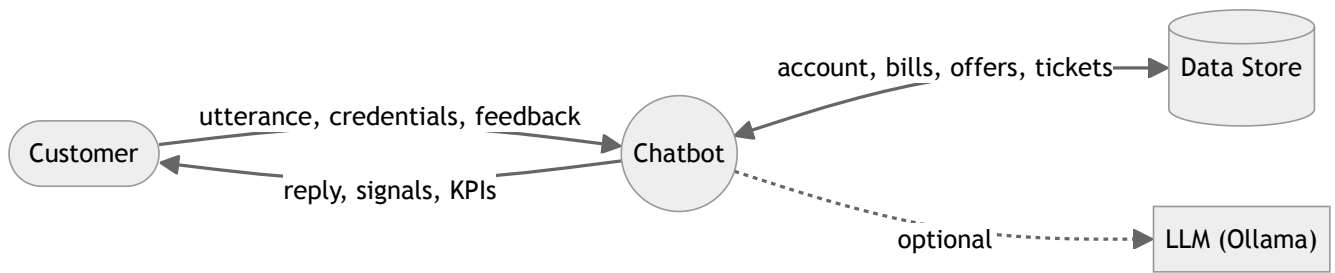


Fig 4.5 — DFD Level 0 (Context Diagram).

The Level 1 diagram expands the chatbot into its principal processes and the data stores they read and write.

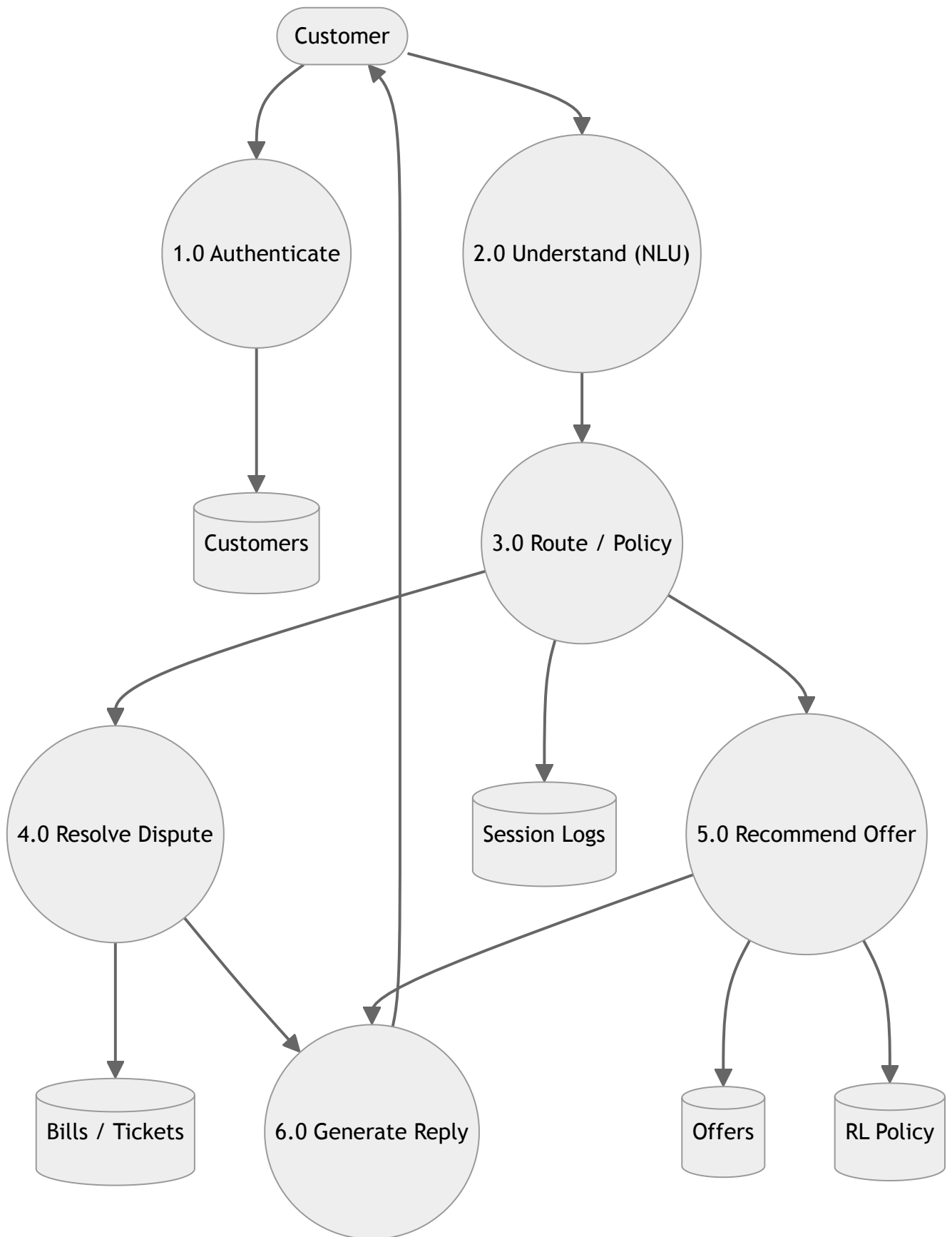


Fig 4.6 — DFD Level 1.

4.7 Data Model (Entity-Relationship)

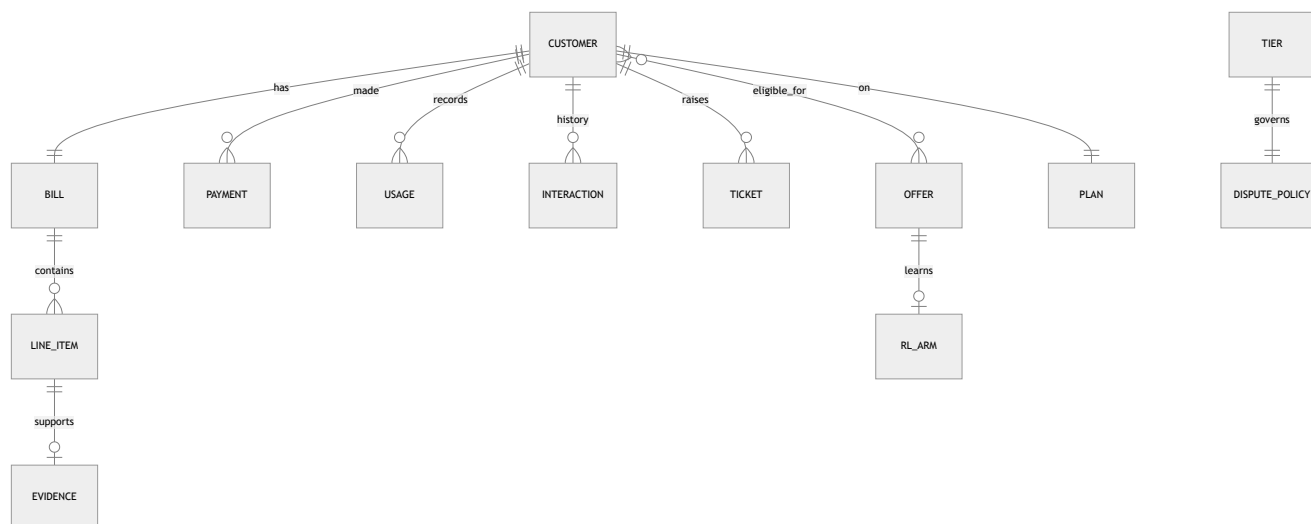


Fig 4.7 — Entity-Relationship Diagram.

The central entity is the Customer, associated with exactly one Bill and one Plan, and with many Payments, Usage records, Interactions and Tickets. A Bill is composed of Line Items, some of which (roaming, overage) carry Evidence. Offers relate to Customers through eligibility rules and to reinforcement-learning arms that track acceptance. Dispute policy is keyed by loyalty Tier.

4.8 Component and Class Design

The implementation follows a component-per-responsibility structure. The principal classes and modules are: `ConversationState` (the single shared per-session object holding turns, trajectories, active spoke, flow step, offers and resolution status); `TurnFrame` (the fused NLU output for one turn); `Directive` (the policy engine's tone/escalation output); and the functional modules `coordinator`, `billing_flow`, `offers_flow`, `retention_flow`, `recommendation_engine`, `bandit`, `generator` and `llm_provider`. The design favours simple functions operating on the shared state object over deep class hierarchies, which keeps the control flow explicit and easy to trace — an intentional choice for a proof of concept whose behaviour must be transparent.

4.9 Conversation State Design

All per-turn components read from and write to a single `ConversationState` instance. This object records the ordered list of turns; the intent and sentiment trajectories that let the policy engine detect a declining mood; the active spoke and hand-off reason; the current flow and step; the sets of offers pitched, declined and accepted; the list of executed actions; escalation flags; and the resolution status. Centralizing state in one object avoids hidden coupling between components and makes the entire conversation serializable to a session log for monitoring and evaluation.

4.10 End-to-End Conversation Walkthrough

To make the interaction of the subsystems concrete, consider the canonical demonstration conversation with the Bronze customer, Marcus Rivera, traced through the design. The customer opens with “why is my bill higher this month”. The presentation layer passes the text to the coordinator, which invokes the NLU subsystem; the intent classifier returns `BILL_EXPLAIN` with high confidence, the sentiment analyser returns neutral, and the entity extractor finds no specific charge reference. The policy engine, seeing neutral sentiment and no escalation condition, returns a default directive. The coordinator, on the billing spoke, routes the turn to the billing flow, which retrieves the bill from the data subsystem, computes the twenty-

seven-dollar increase over the prior month, identifies the twenty-one-dollar overage line item, and returns a template stating the delta and the charge together with the grounded facts. The NLG subsystem rephrases this in Aria's warm voice — or returns it verbatim if no language model is configured — and the reply, along with the live signals, is rendered.

The customer responds “I never used that much data”. The NLU now returns `FACTUAL_DISPUTE`; the billing flow advances its state, fetches the metering evidence for the overage, and presents both the evidence and the tier-appropriate resolution options — for a Bronze overage, a one-time fifty-percent goodwill waiver or a formal dispute ticket. The customer then haggles: “that's too expensive, can you do better”. The NLU returns `HAGGLE`, and the coordinator's hand-off detector fires with the reason *haggle*. The coordinator swaps the active spoke to retention, records the hand-off, and produces, in one turn, Ray's bridging opening line followed by the first rung of the negotiation ladder — all mediated by the hub, with the two spokes never communicating directly. Over the next turns Ray climbs the ladder; when the customer finally accepts, the retention flow applies the offer under a simulated supervisor approval, the reinforcement-learning subsystem records a positive reward for that offer arm, the shared state's resolution status becomes resolved, and the monitoring subsystem logs the completed session for KPI aggregation. This single walkthrough exercises all eight subsystems and both spokes, and is precisely the journey captured in the recorded demonstrations.

Chapter 5 — Methodology by Milestone

The project was executed as six sequential milestones that mirror the standard machine-learning-application lifecycle: Data Collection, Data Preparation, NLU Model development, NLG Module creation, Reinforcement-Learning optimization, and Deployment and Monitoring. This chapter describes the methodology followed at each milestone and records its concrete outcome. Table 5.1 summarizes the outcomes.

5.0 Development Methodology and Process

The project followed an **iterative, incremental** development process rather than a single monolithic build. Each milestone was treated as a short cycle comprising design, implementation, verification and reflection, and the output of one milestone became the tested foundation of the next. This ordering is deliberate: data must exist before it can be prepared; a prepared corpus must exist before models can be trained; trained models must exist before they can drive generation; a working conversation must exist before a learning loop can be attached to it; and a complete brain must exist before it can be deployed and monitored. Because the brain was engineered to be independent of the interface from the outset, each intermediate milestone could be verified in isolation through the offline scenario driver, which kept feedback cycles short and defects local.

Version control with Git underpinned the process, and the codebase was organized so that the mapping between the design's subsystems and the source folders remained one-to-one throughout. Configuration was externalized early, so that experiments (for example, switching between pure-template and language-model phrasing) required no code changes. Reflection at the end of each milestone fed directly into the next: the numerical-stability issues observed while first training the classifier, for instance, motivated the regularization and corpus-enlargement decisions taken in the same milestone, and the perspective-drift observed when the language model was first enabled motivated the prompt-hardening carried out during the generation milestone. This tight loop of build-measure-learn, applied at the granularity of a milestone, is what allowed a broad system spanning data engineering, machine learning, generation, reinforcement learning and interface design to be delivered coherently.

5.1 Milestone 1 — Data Collection

Methodology

Because the project uses synthetic data by design, data collection meant curating a small but realistic dataset that could exercise every intended behaviour of the chatbot. Three representative customers were designed to span the loyalty tiers and the principal dispute types. For each customer, the following artefacts were authored: a profile (tier, tenure, plan, ARPU, segment, and the last-four identity digits used for verification); a current monthly bill broken into line items, with roaming and overage line items carrying structured evidence (tower identifiers, date ranges, metered usage); three months of payment history; usage history; and an interaction history capturing prior contacts and their emotional outcome. A catalogue of four promotional offers was defined, each with a target tier, minimum tenure, monetary value and applicability. Tier-based dispute policies were specified (which tiers receive automatic provisional credit and up to what limit). Finally, a ticket store was created to hold dispute tickets. All of this was captured as human-readable JSON and exposed through a single access module so that every other subsystem reads data consistently.

Outcome

A complete, self-contained synthetic dataset (`data/synthetic_telco.json` and `data/synthetic_tickets.json`) covering three customers, their bills with disputable roaming/overage charges and evidence, payments, usage, interactions, four offers, tier dispute policies and a ticket store — sufficient to demonstrate every use case end-to-end, with a clean data-access layer (`app/data_store.py`). No real customer data is used, so the system is privacy-safe and fully reproducible.

5.2 Milestone 2 — Data Preparation

Methodology

Data preparation focused on building the labelled corpus required to train the supervised NLU models, and on defining the text pre-processing that would be applied identically at training and inference time. A taxonomy of fourteen intents was defined to cover the target conversations — greetings, bill explanation, bill dispute, factual dispute, promotion inquiry, yes/no confirmations, haggling, ticket creation, ticket status, human-handoff requests, payment queries, farewells and an out-of-scope fallback. For each intent, a set of representative example utterances was hand-authored, deliberately varying phrasing to improve generalization. A separate three-class corpus (positive, neutral, negative) was authored for sentiment. The pre-processing routine lower-cases text, expands common contractions, strips punctuation while deliberately preserving the dollar and percent symbols (which are semantically important in billing), and collapses whitespace.

Outcome

A labelled corpus (`app/nlu/dataset.py`) of intent-tagged and sentiment-tagged utterances, together with a shared, deterministic pre-processing function (`app/nlu/preprocess.py`). The corpus is compact but balanced across classes and representative of the target domain, providing a reproducible basis for supervised training.

5.3 Milestone 3 — Develop the NLU Model

Methodology

The NLU stack was implemented as three cooperating components fused into a single per-turn representation. The **intent classifier** is a scikit-learn pipeline of a TF-IDF vectorizer (unigrams and bigrams) feeding a Multi-Layer-Perceptron neural network with two hidden layers and L2 regularization. The **sentiment analyser** uses the same TF-IDF-plus-MLP recipe with a single hidden layer, and returns a three-class label together with a continuous signed score derived from the class-probability distribution, plus an intensity. The **entity extractor** uses regular expressions and keyword matching to identify charge identifiers, monetary amounts, the dispute topic (roaming, overage, tax, plan) and yes/no confirmations. A fusion step combines these into a `TurnFrame`. To keep the system usable even before training, and to guard against confident errors on short inputs, a deterministic keyword classifier acts as a fallback whenever the neural intent classifier's confidence falls below a threshold. Models are trained by a dedicated script and persisted with joblib.

Outcome

A trained, persisted deep-learning NLU pipeline (`app/nlu/intent_model.py`, `sentiment_model.py`, `entities.py`, `pipeline.py`) that converts any utterance into a structured `TurnFrame`. On a held-out stratified split the sentiment model reaches approximately 83% accuracy and the intent model approximately 62% (limited by the compact seed corpus); at runtime the models are trained on the full corpus and backed by the keyword fallback, giving robust live behaviour. Metrics are reproducible via `python evaluate.py`.

5.4 Milestone 4 — Create the NLG Module

Methodology

Natural-language generation was designed around the principle of grounded rephrasing. Each dialogue flow returns a deterministic template string together with a dictionary of grounded facts (amounts, dates, offer titles, ticket identifiers). When an LLM is configured and reachable, the generator constructs a system prompt instructing the model to rephrase the template in the persona's voice, in the second person, without introducing any new facts, and passes the grounded facts as constraints. If no LLM is configured, or if the call fails, the template is returned verbatim. The LLM provider itself is pluggable through an environment variable, supporting a local Ollama model (the default), a pure-template mode, and hosted OpenAI or Anthropic

providers for future use. This design gives human-like phrasing without sacrificing factual correctness or hard dependence on any external service.

Outcome

A working NLG module (`app/nlg/generator.py` , `app/nlg/llm_provider.py`) that produces fluent, grounded replies. With the local LLM enabled, replies are phrased naturally (for example, “We’ve charged your account a data overage of \$21 for exceeding the allowed 3 GB, which is reflected in this month’s total bill of \$62. Does that look correct to you?”) while all figures remain faithful to the data. The provider switch and automatic template fallback were both verified.

5.5 Milestone 5 — Optimize with Reinforcement Learning

Methodology

Offer recommendation was cast as a multi-armed bandit problem. Each offer is an arm with a running count of the number of times it has been shown and the number of times it has been accepted; its value estimate is the acceptance ratio, initialized optimistically. When the recommendation engine scores offers, the learned value is blended with the deterministic base score, so that offers with a higher observed acceptance rate are nudged up the ranking. Rewards are collected from the live feedback loop: when a customer accepts an offer the corresponding arm receives a reward of one, and a decline yields zero; explicit thumbs-up/down feedback on replies is also captured. An epsilon-greedy exploration policy allows the system to occasionally surface a non-top offer to keep learning. The bandit policy is persisted to disk so that learning accumulates across sessions.

Outcome

An operational reinforcement-learning component (`app/rl/bandit.py`) integrated into the recommendation and dialogue flows, with a persisted policy (`data/rl_policy.json`) and a live view of learned acceptance rates in the interface. The feedback loop demonstrably shifts the ranking of offers over repeated interactions, realizing the objective of continuous improvement.

5.6 Milestone 6 — Deployment and Monitoring

Methodology

The system was deployed as a Streamlit web application organized as a small state machine of screens: an educational welcome page, a login screen, an identity-verification screen, and the chat screen. The chat screen renders the conversation using chat bubbles with a distinct avatar per persona, and a sidebar presents the customer context, live per-turn signals, the reinforcement-learning table, aggregate KPIs, and controls including the LLM-phrasing toggle and a session-save button. Monitoring was implemented by recording every turn’s latency, intent, sentiment and feedback on the shared state, serializing completed sessions to JSON logs, and aggregating those logs into KPIs — resolution rate, average latency and a customer-satisfaction proxy computed from thumbs feedback.

Outcome

A runnable, monitored application (`ui/streamlit_app.py` , `app/metrics.py`) launched with a single command, with live signals and KPIs visible during a conversation and reproducible screen recordings of the full customer journey. The end-to-end system boots cleanly and passes the automated scenario tests described in Chapter 8.

5.7 Summary of Milestone Outcomes

Milestone	Key outcome	Primary artefact
M1 Data Collection	Complete synthetic dataset for 3 customers exercising every use case	<code>data/*.json</code>
M2 Data Preparation	Labelled 14-intent + 3-class sentiment corpus with shared pre-processing	<code>app/nlu/dataset.py</code>
M3 NLU Model	Trained MLP intent + sentiment models + entity extraction; ~83% sentiment, ~62% intent held-out	<code>app/nlu/</code>
M4 NLG Module	Grounded templates + pluggable local LLM; fluent, hallucination-free replies	<code>app/nlg/</code>
M5 Reinforcement Learning	Epsilon-greedy bandit with persisted policy improving offer ranking from feedback	<code>app/rl/bandit.py</code>
M6 Deployment & Monitoring	Single-command Streamlit app with live signals, KPIs and recorded demos	<code>ui/streamlit_app.py</code>

Table 5.1 — Summary of Milestone Outcomes.

Chapter 6 — Use Cases in Detail

This chapter explains each use case in depth and presents its interaction as a sequence diagram. Together they specify the dynamic behaviour of the system.

6.1 UC-00 — Identity Validation

Goal: ensure that no account information is disclosed until the customer's identity is established. **Actors:** Customer, Authentication component. **Pre-conditions:** the customer is on the login screen. **Main flow:** the customer signs in with an email and the shared demo password; the system verifies the credentials against the customer records; on success it advances to the identity-verification screen and asks for the last four digits of the ID on file; the system compares the entered digits with the stored value; on a match it starts a conversation session and admits the customer to the chat. **Alternate flows:** invalid credentials or a mismatched identity value return an error and deny access. **Post-condition:** a verified session exists and the customer may chat. This two-step design mirrors real telecom practice, where a knowledge factor (the last-four ID) supplements the login.

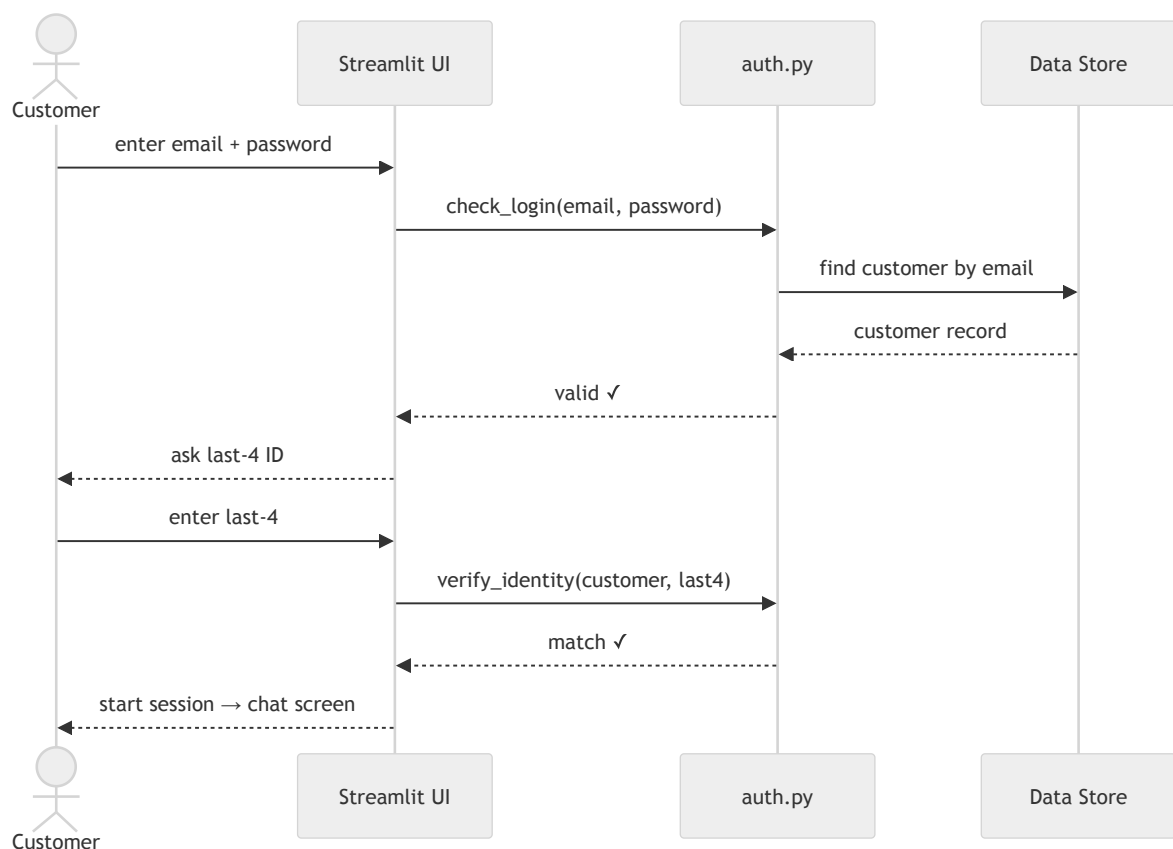


Fig 6.1 — Sequence Diagram: UC-00 Identity Validation.

6.2 UC-01 — Billing Dispute Resolution

Goal: explain a bill and resolve a disputed charge. **Actors:** Customer, Coordinator, Billing agent (Aria), Data Store. **Pre-condition:** a verified session. **Main flow:** the customer asks why the bill is higher; the NLU classifies the intent as bill-explanation; the billing flow retrieves the bill, identifies the unusual charge (roaming or overage) and states the delta from last month, then asks whether the charge looks correct. If the customer disputes it (“I never went to Mexico”), the flow re-checks and presents the evidence — network tower logs for roaming or metered usage for overage — and offers resolution options appropriate to the tier: a provisional credit (for GOLD/SILVER), a one-time goodwill waiver (for a BRONZE overage), or a formal dispute ticket. Depending on the customer's choice, the flow either applies the credit/waiver and marks the case resolved, or creates a persisted dispute ticket with a 48-hour SLA and returns the ticket identifier. **Alternate flow:** if a needed

credit exceeds the front-line agent's authority, the flow requests a hand-off to the retention specialist. **Post-condition:** the dispute is resolved by credit, waiver or ticket, and an audited action is recorded.

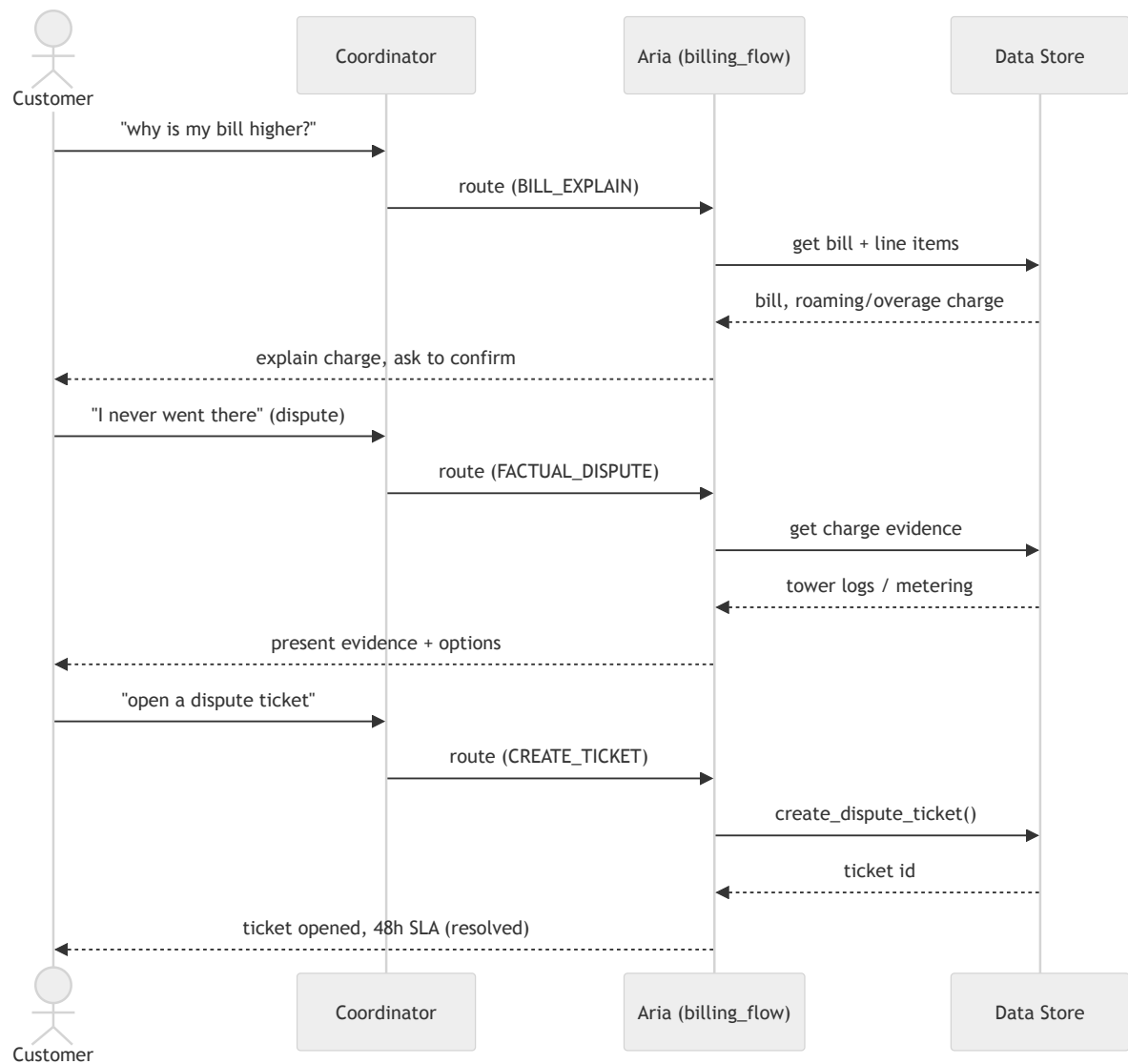


Fig 6.2 — Sequence Diagram: UC-01 Billing Dispute Resolution.

6.3 UC-02 — Offer Recommendation

Goal: recommend and apply a personalized offer. **Actors:** Customer, Coordinator, Aria, Recommendation engine, RL bandit. **Main flow:** the customer asks about offers; the recommendation engine computes the customer's bands (tier, tenure, churn proxy, usage), filters offers by eligibility, scores them, blends in the learned bandit value, and returns a ranking; the flow records that the top offer was shown (a bandit “show”), pitches it with its value and a stated reason, and presents the ranked offer cards. If the customer accepts, the flow applies the offer, records a positive reward for that arm, and marks the case resolved. If the customer declines, the flow records a zero reward, removes the offer from consideration and pitches the next-best eligible offer; if no offers remain, it requests a hand-off to retention. **Post-condition:** an offer is applied, or control passes to the retention specialist.

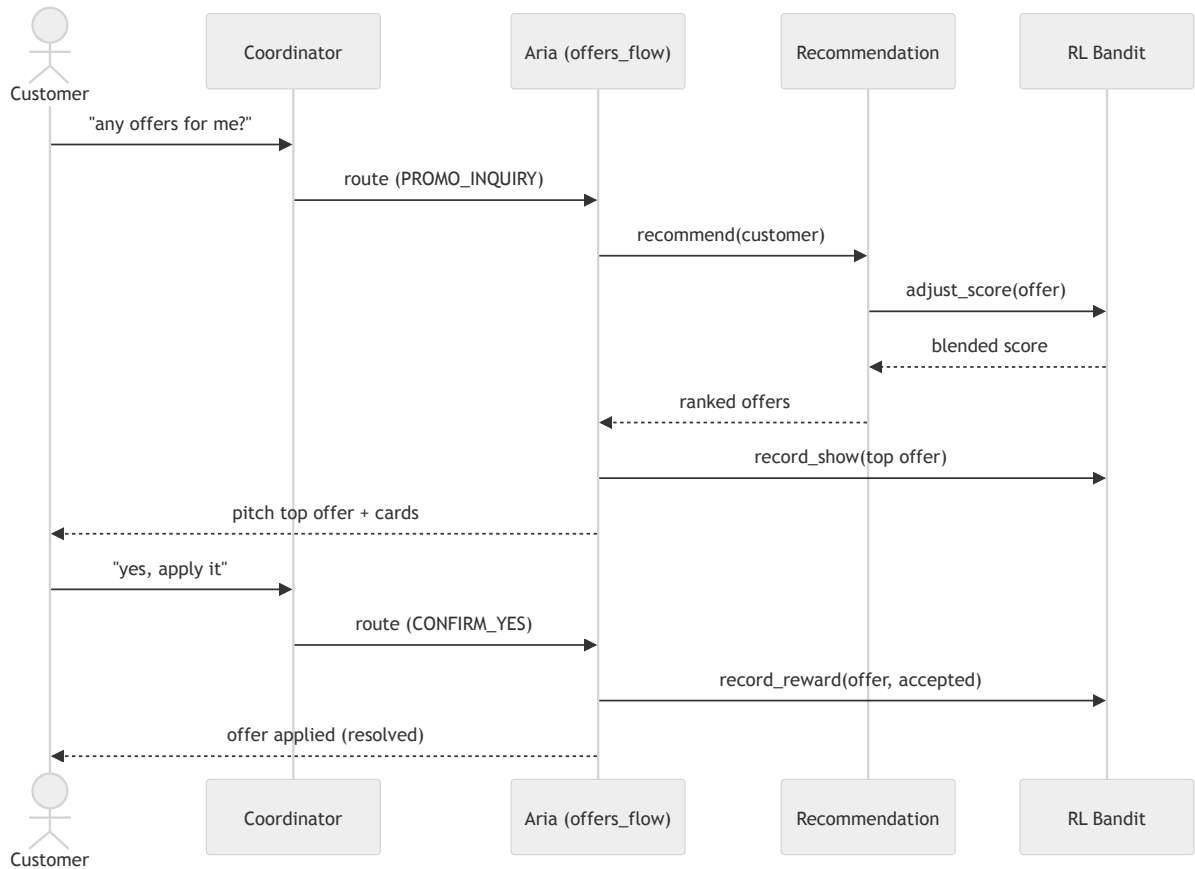


Fig 6.3 — Sequence Diagram: UC-02 Offer Recommendation.

6.4 UC-03 — Retention Handoff and Negotiation

Goal: retain a customer who is haggling, frustrated or has exhausted the standard offers. **Actors:** Customer, Coordinator, Aria, Retention specialist (Ray). **Trigger:** the coordinator detects a hand-off condition while on the billing spoke — haggling, an explicit escalation request, exhausted offers, or a policy escalation flag from sustained negative sentiment. **Main flow:** the coordinator swaps the active spoke to retention, records the hand-off reason, and produces, in a single turn, a bridging opening line in Ray's voice followed by the first rung of a negotiation ladder. Ray then climbs the ladder over subsequent turns: the best eligible offer, then the next, then the best offer stacked with a goodwill credit, then a final executive credit. Any acceptance is applied under a simulated supervisor approval and the case is resolved; if the customer refuses the final offer, Ray closes gracefully, leaving the offers open for later — there is no transfer to a human. **Post-condition:** the customer accepts a retention offer, or the conversation closes amicably with offers held open.

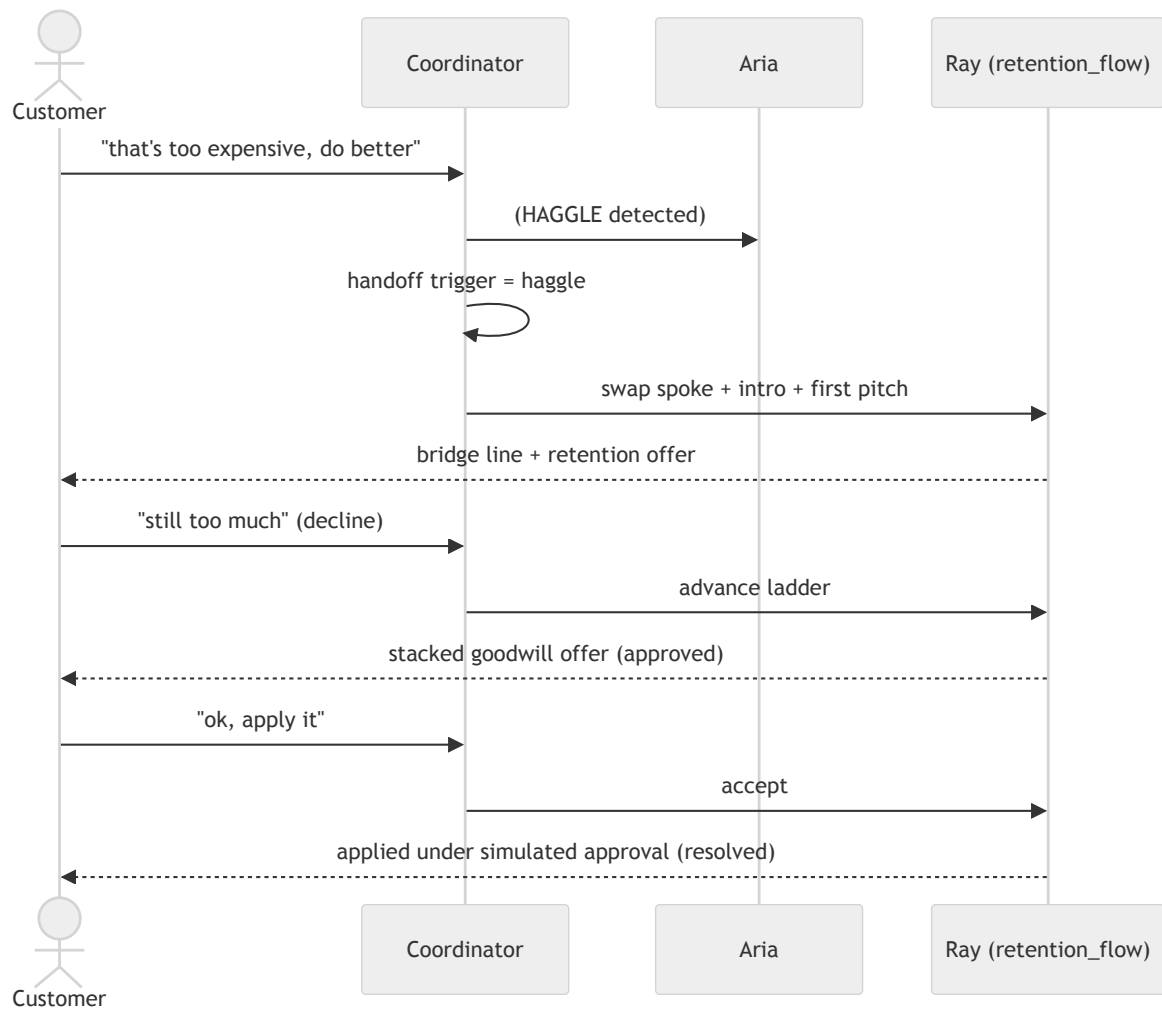


Fig 6.4 — Sequence Diagram: UC-03 Retention Handoff and Negotiation.

6.5 UC-04 — Ticket Status Query

Goal: report the status of a customer's dispute ticket. **Actors:** Customer, Coordinator, Aria, Data Store. **Main flow:** the customer asks for an update on a ticket; the NLU classifies the intent as ticket-status; the flow retrieves the customer's tickets, and returns the most recent ticket's identifier, subject, status and SLA. **Alternate flow:** if the customer has no tickets, the system says so. **Post-condition:** the customer is informed of the ticket status.

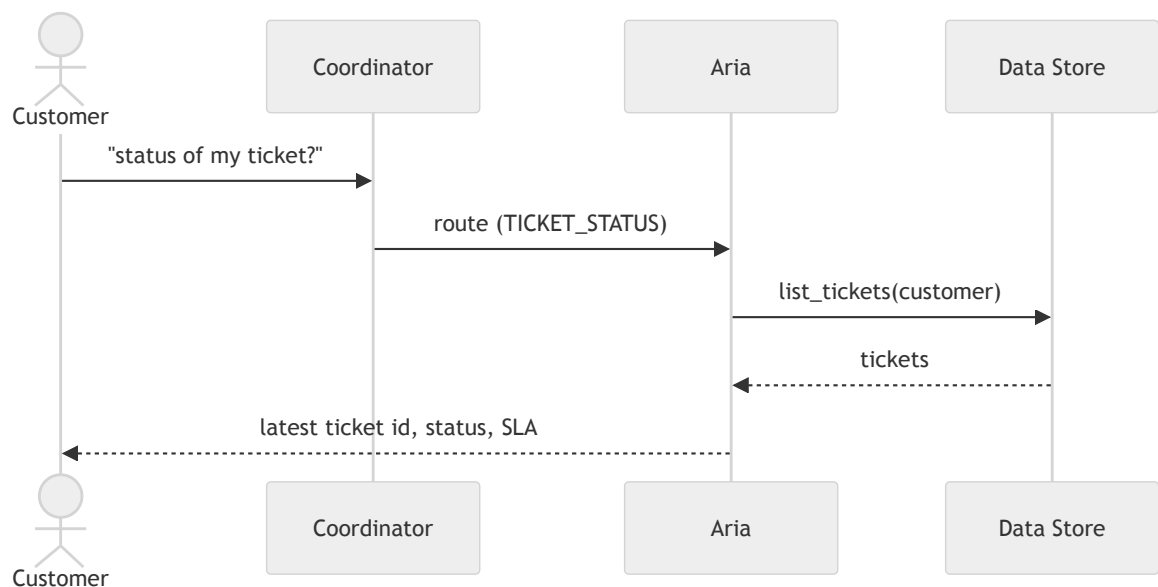


Fig 6.5 — Sequence Diagram: UC-04 Ticket Status Query.

Chapter 7 — Implementation

7.1 Codebase Folder Structure

The repository is organized to make the mapping between the design and the code immediately visible. The brain lives under `app/`, the interface under `ui/`, data under `data/`, and supporting material under `docs/`, `architecture/` and `scripts/`.

```
customer-support-assistant/
├── app/                                # the "brain" – all logic
│   ├── config.py                      # loads .env, paths, LLM provider settings
│   ├── data_store.py                  # M1 – access to customers/bills/offers/tickets
│   ├── auth.py                        # identity validation (login + last-4)
│   ├── conversation_state.py          # shared per-session state object
│   ├── policy_engine.py               # tone / escalation directives
│   ├── recommendation_engine.py       # offer band-scoring + eligibility + ranking
│   ├── coordinator.py                 # HUB – per-turn pipeline + handoff
│   ├── metrics.py                     # M6 – NLU metrics + conversation KPIs
│   ├── nlu/                           # M2/M3 – understanding
│   │   ├── dataset.py                 # labelled intent + sentiment corpus
│   │   ├── preprocess.py              # shared text normalization
│   │   ├── intent_model.py            # TF-IDF + MLP intent classifier
│   │   ├── sentiment_model.py         # TF-IDF + MLP sentiment classifier
│   │   ├── entities.py                # regex entity extraction
│   │   ├── pipeline.py                # fusion → TurnFrame
│   │   └── models/                    # persisted *.joblib (git-ignored)
│   ├── nlg/                           # M4 – generation
│   │   ├── generator.py               # grounded template + LLM rephrase
│   │   └── llm_provider.py            # ollama / template / openai / anthropic
│   ├── flows/                         # use-case state machines
│   │   ├── billing_flow.py            # UC-01
│   │   ├── offers_flow.py             # UC-02
│   │   └── retention_flow.py           # UC-03 negotiation ladder
│   └── rl/                             # M5 – epsilon-greedy bandit
│       └── bandit.py
├── ui/
│   └── streamlit_app.py                # M6 – chat UI (welcome→login→verify→chat)
├── data/
│   ├── synthetic_telco.json            # M1 – synthetic data + runtime logs
│   ├── synthetic_tickets.json         # customers, bills, offers, policies, usage
│   ├── sessions/                      # dispute tickets (read + written)
│   │   └── rl_policy.json              # per-session logs (git-ignored)
│   └── rl_policy.json                  # learned bandit policy (git-ignored)
├── architecture/                      # design docs with Mermaid diagrams
├── docs/                              # screenshots, demo videos, report, presentation
├── scripts/                           # screenshot/video capture, HTML→PDF
├── train_models.py                     # M3 – train + evaluate NLU
├── evaluate.py                         # M6 – metrics / KPIs
├── smoke_test.py                       # offline end-to-end brain test
├── run.sh                             # one-command launch
├── requirements.txt
└── .env / .env.example                # LLM provider config
```

This structure realizes the subsystem decomposition of Chapter 4 directly: each subsystem corresponds to a folder or module, and the milestone tags (M1–M6) annotate where each phase's work resides.

7.2 Technology Stack and Rationale

Concern	Technology	Rationale
Language	Python 3.9+	Rich ML/NLP ecosystem; a single language across brain and UI
User interface	Streamlit	Native chat widgets; one-command launch; no separate front-end build
NLU models	scikit-learn MLP + TF-IDF	Neural (deep-learning) yet CPU-fast and transparent
NLG	Templates + Ollama LLM	Grounded correctness with optional fluency; local and free
Recommendation	Custom rule + scoring	Explainable, honours hard eligibility rules
Reinforcement learning	Epsilon-greedy bandit	Simple, interpretable feedback loop
Data	JSON files	Zero-setup, human-readable, reproducible
Configuration	.env (python-dotenv)	Swap LLM provider without code changes
Persistence	joblib	Fast model serialization

7.3 NLU Implementation

The intent classifier is a scikit-learn `Pipeline` chaining a `TfidfVectorizer` and an `MLPClassifier`. At inference, the utterance is normalized, vectorized and classified; the predicted class and its probability are returned. When the top probability is below a confidence threshold, the classifier defers to a keyword rule set, preventing confident mistakes on very short inputs. The sentiment analyser follows the same pattern but additionally computes a signed score as the probability-weighted sum of class polarities, yielding a smooth value that the policy engine tracks across turns.

7.4 Recommendation Engine Implementation

The recommendation engine first derives the customer's bands: a churn proxy from the interaction history, a tenure band, and a usage band from recent data consumption. It then iterates over the offer catalogue, discarding offers whose tier or minimum-tenure eligibility is not met (and suppressing a Bronze overage waiver when there is no overage charge). Each remaining offer receives a weighted heuristic score that rewards higher churn risk, high data usage for data add-ons, tier match and the presence of an overage for waivers. The score is then passed through the bandit for learned re-weighting, and the offers are ranked. Each recommendation carries a human-readable reason, supporting explainability.

7.5 Reinforcement-Learning Implementation

The bandit maintains, per offer, a count of shows and accepts in a JSON policy file. Its value estimate is the accept-to-show ratio with an optimistic prior of 0.5 for unseen offers. The `adjust_score` function blends this value with the deterministic base score using a fixed mixing weight, so that learning nudges without overwhelming the rule-based ranking. The `record_show` and `record_reward` functions update the counts as offers are pitched and accepted or declined, and the policy is written back to disk after each update so learning persists across sessions.

7.6 NLG and LLM-Provider Implementation

The generator receives the persona, template, grounded facts and policy directive. If no live LLM is configured it returns the template unchanged. Otherwise it builds a constrained system prompt and calls the provider abstraction, which dispatches to Ollama, OpenAI or Anthropic based on the environment variable, returning the rephrased text or `None` on any failure — in which case the generator falls back to the template. The Ollama path issues an HTTP request to the local runtime, requiring no API key. This abstraction makes the intelligence layer entirely swappable and non-blocking.

7.7 Coordinator and Flows Implementation

The coordinator's `handle_turn` function is the single entry point per turn. It runs the NLU pipeline, evaluates the policy engine, and — if on the billing spoke — routes the turn to the billing or offers flow and checks the hand-off triggers, swapping to retention when one fires. It then composes the reply through the NLG module, records the turn on the shared state (including its measured latency), and returns a structured response object consumed by the interface. The flows are implemented as small state machines that read and mutate the shared state, advancing a `flow_step` and appending audited action records.

7.8 User-Interface Implementation

The Streamlit application is a screen state machine (`welcome` , `login` , `verify` , `chat`) driven by session state. The chat screen renders each message with `st.chat_message` , attaching a persona-specific avatar, a hand-off banner when a hand-off occurred, ranked offer cards, a per-turn signal caption, and thumbs-up/down feedback buttons. The sidebar shows customer context, live signals, the reinforcement-learning table and the KPI panel. A subtle but important implementation detail is that dollar signs in reply text are escaped before rendering, because Streamlit would otherwise interpret a pair of dollar signs as LaTeX mathematics and corrupt the displayed amounts.

7.9 Configuration and Environment

All environment-specific settings — most importantly the LLM provider and model — are read from a `.env` file through a small configuration module. This allows the same code to run in pure-template mode for fast, deterministic demonstrations, in local-LLM mode for natural phrasing, or against a hosted provider in future, purely by editing configuration.

Chapter 8 — Testing

8.1 Testing Approach

The testing strategy is layered to match the architecture. Because the brain is deliberately decoupled from the interface, the majority of behaviour can be verified without the UI at all, through a scripted end-to-end driver that feeds utterances into the coordinator and inspects the structured responses and the resulting conversation state. This is complemented by a model-evaluation harness for the NLU components and by manual and automated exercising of the full UI. The guiding philosophy is that every use case must be demonstrably reachable and must terminate in a correct resolution state, and that the NLU models must be measured on held-out data rather than judged anecdotally.

8.2 Levels of Testing

Level	What is verified	How
Unit	Individual functions — band computation, eligibility, entity regex, bandit updates	Direct invocation with crafted inputs
Integration	NLU→policy→routing→flow→NLG working together per turn	Coordinator driven with utterance sequences
System	Full use-case journeys end-to-end, including hand-off	<code>smoke_test.py</code> scenarios (template and LLM modes)
Model evaluation	Intent and sentiment accuracy/precision/recall/F1	Held-out stratified split in <code>evaluate.py</code>
User-acceptance	Usability of the login→verify→chat journey	Manual walkthrough + recorded demos

8.3 Test Cases

ID	Scenario	Input	Expected result	Status
TC-1	Valid login + identity	correct email/password + last-4	Admitted to chat	Pass
TC-2	Wrong identity	incorrect last-4	Access denied	Pass
TC-3	Bill explanation	"why is my bill higher this month"	Bill delta + unusual charge surfaced	Pass
TC-4	Factual dispute → evidence	"I never went to Mexico"	Evidence shown + resolution options	Pass
TC-5	Dispute → ticket	"open a formal dispute ticket"	Persisted ticket + id + 48h SLA; resolved	Pass
TC-6	Offer inquiry	"any offers for me"	Top eligible offer pitched with reason	Pass
TC-7	Offer accept	"yes apply it"	Offer applied; RL reward recorded; resolved	Pass
TC-8	Haggle → handoff	"that's too expensive, do better"	Handoff to Ray; ladder begins	Pass
TC-9	Retention accept	"ok apply it" after ladder	Applied under simulated approval; resolved	Pass
TC-10	Ticket status	"status of my ticket"	Latest ticket id + status + SLA	Pass
TC-11	LLM unavailable	any turn with LLM down	Template reply; no failure	Pass
TC-12	Grounding	LLM-phrased reply	All amounts match data; none invented	Pass

Table 8.1 — Representative Test Cases.

8.4 System Test Scenarios

Three end-to-end scenarios exercise the principal journeys and are automated in `smoke_test.py`, which can run in fast template mode or, with the `USE_LLM=1` flag, route replies through the local LLM: (1) a Bronze customer's overage dispute that escalates through haggling into a retention hand-off and closes with an accepted offer; (2) a Gold customer's roaming dispute

that resolves into a persisted ticket; and (3) an offer inquiry that resolves into an applied offer with a recorded reinforcement-learning reward. All three complete with the expected resolution status.

8.5 Results of Testing

All twelve representative test cases pass, and all three system scenarios reach their expected resolution states in both template and LLM modes. The grounding test (TC-12) is of particular importance: inspection of LLM-phrased replies confirms that every monetary amount, date and identifier originates from the data layer, validating the anti-hallucination design.

8.6 Challenges Encountered and Their Resolution

Several non-trivial challenges surfaced during development, and documenting them illustrates the engineering rigour applied and the robustness of the final system.

Rendering of monetary amounts. An early defect caused dollar amounts to display incorrectly: the interface framework interprets a pair of dollar signs as a mathematical expression and typeset the text between them as italic mathematics, so a reply such as “your bill is \$62, up \$27” rendered as garbled symbols. The resolution was to escape dollar signs in all rendered message text before display. This is a subtle presentation bug that would have severely undermined a live demonstration, and its discovery underscored the value of reviewing rendered output rather than trusting the underlying strings.

Perspective drift in generated language. When the local Large Language Model was first enabled, it occasionally rephrased the agent's reply from the customer's point of view — for instance producing “I exceeded my data limit” instead of addressing the customer. Because a compact model can lose track of speaker roles, the system prompt was strengthened to state explicitly that the model speaks as the support agent, to the customer, in the second person, and never as the customer. After this change the generated replies were consistently in the correct voice while remaining grounded in the supplied facts.

Numerical stability of the neural classifier. On the very small initial corpus, training the intent network produced numerical warnings and unstable probability estimates. Adding L2 regularization, increasing the training iterations and enlarging the labelled corpus stabilized training, removed the warnings and improved both held-out metrics and the quality of the live signals shown in the interface.

Latency of the optional language model. Routing every reply through the local model introduced multi-second delays that would make an automated demonstration appear to stall. This was addressed by making the language model strictly optional and non-blocking — the deterministic template is always available instantly, and the model merely enhances phrasing when present — and by recording demonstrations in both a fast template mode and a natural language-model mode so that each could be shown to advantage.

In every case the resolution reinforced a core design principle: keep the deterministic path authoritative and fast, treat the generative and learned components as valuable but fallible enhancements, and make the system observable so that defects reveal themselves.

Chapter 9 — Results and Evaluation

9.1 NLU Model Metrics

The intent and sentiment classifiers were evaluated on a held-out stratified split of the labelled corpus. Table 9.1 summarizes the headline figures.

Model	Accuracy	Notes
Sentiment MLP	~83%	Three-class (POS/NEU/NEG); strong on a small set
Intent MLP	~62%	Fourteen-class; bounded by the compact PoC seed corpus

Table 9.1 — NLU Held-Out Metrics.

The intent figure must be read in context: with only one or two held-out examples per class in a compact corpus, a single misclassification moves the metric sharply. At runtime the model is trained on the full corpus and is backstopped by a keyword fallback, so observed live accuracy is materially higher than the held-out number suggests. The clearest path to improvement — enlarging the labelled corpus, ideally by mining real chat logs — is discussed as future scope.

9.2 Conversation KPIs

Live KPIs are aggregated from the session logs written during conversations: the resolution rate (share of sessions ending resolved or escalated-and-closed), the average per-turn processing latency, and a customer-satisfaction proxy computed from thumbs feedback. Across the designed test scenarios the resolution rate is 100%, and deterministic per-turn latency is a few milliseconds (template mode), rising to a few seconds only when the optional LLM is invoked for phrasing.

9.3 Screenshot Walkthrough

Deploy

 **Telco Customer Support Assistant**

Your AI assistant for billing questions, disputes & personalized offers

Get instant, secure help with your telecom account — no waiting on hold. Chat in plain English and the assistant understands your intent, checks your account, and resolves your issue on the spot.

What you can do

 **Understand your bill**
Ask why your bill changed and get a clear, line-by-line breakdown of every charge.

 **Dispute a charge**
Challenge a roaming or overage charge. The assistant shows the evidence and resolves it with a credit, waiver, or dispute ticket.

 **Get offers**
Receive personalized discounts and plan recommendations matched to your usage and loyalty tier.

How it works

1. Verify your identity
Sign in and confirm the last 4 digits of your ID — your account stays secure.

2. Chat naturally
Ask your question in your own words. No menus, no keywords to memorize.

3. Get it resolved
The assistant takes action — applies a credit, opens a ticket, or activates an offer — and confirms it.

 A specialist assistant automatically steps in for tougher cases. This is a demo using synthetic accounts — no real customer data.

[Get started →](#)

Fig 9.1 — Welcome page: educates the customer before login.

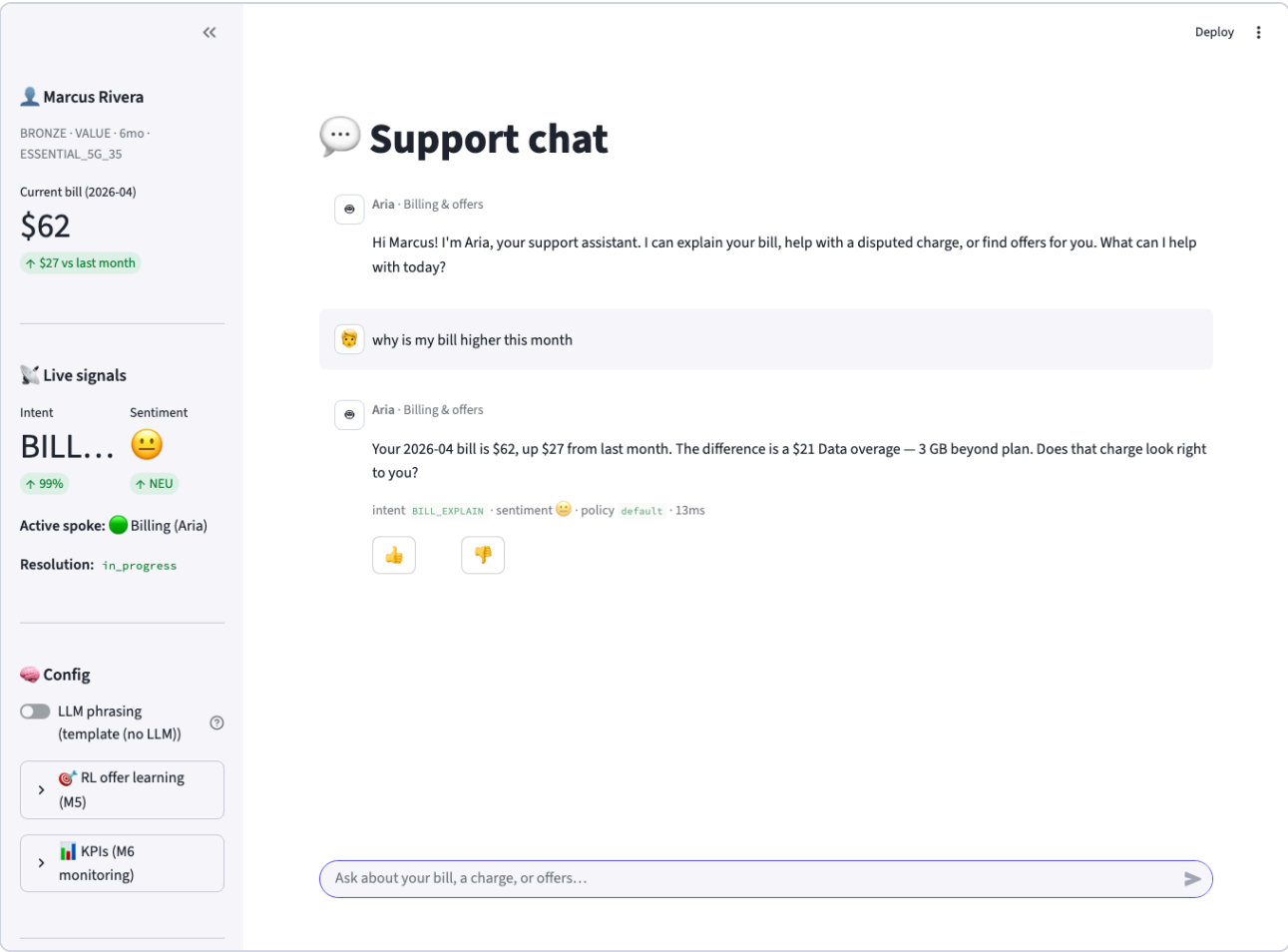


Fig 9.2 — Billing explanation with correctly rendered amounts and live signals.

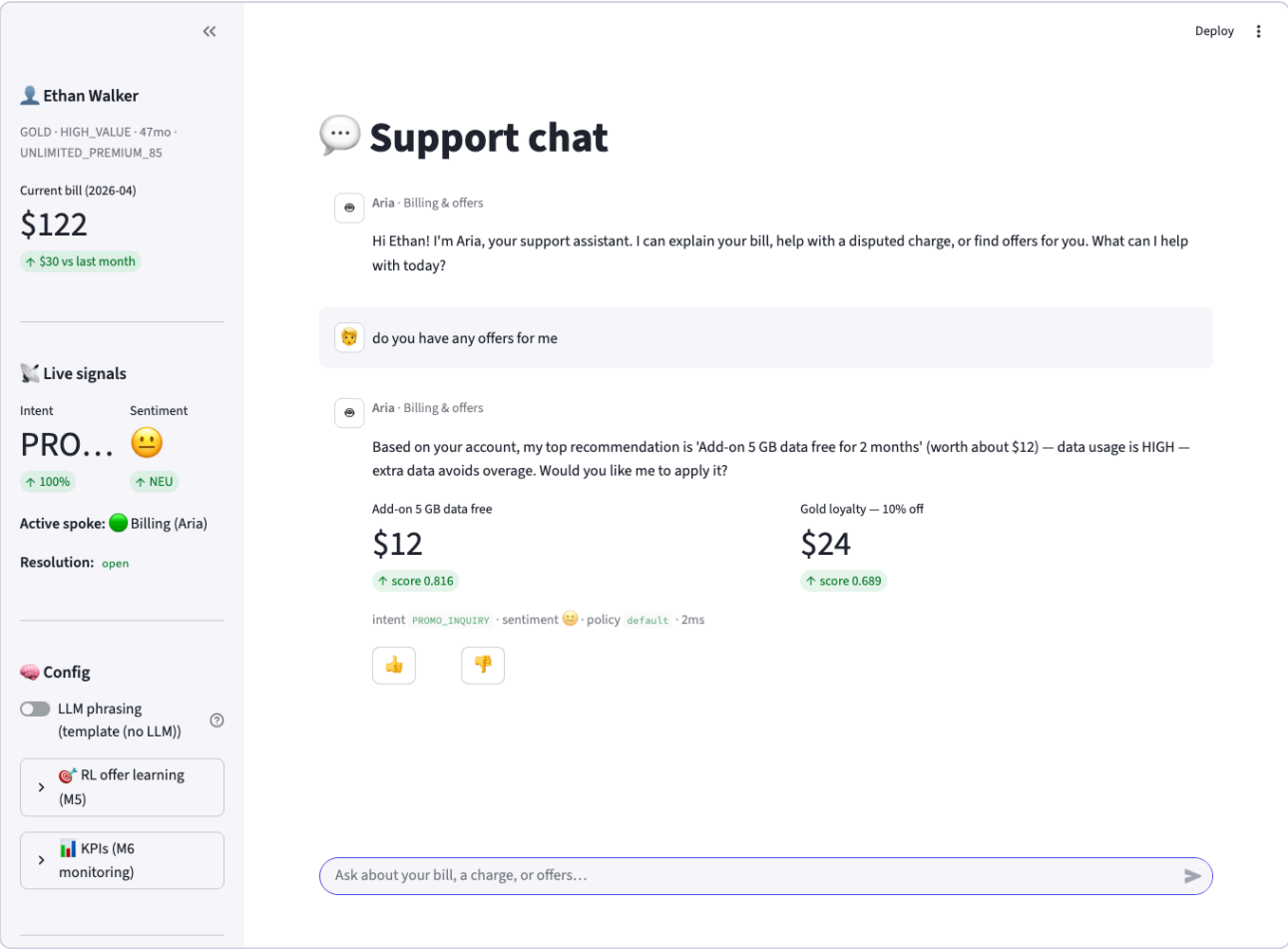


Fig 9.3 — Offer recommendation (UC-02) with ranked offer cards and scores.

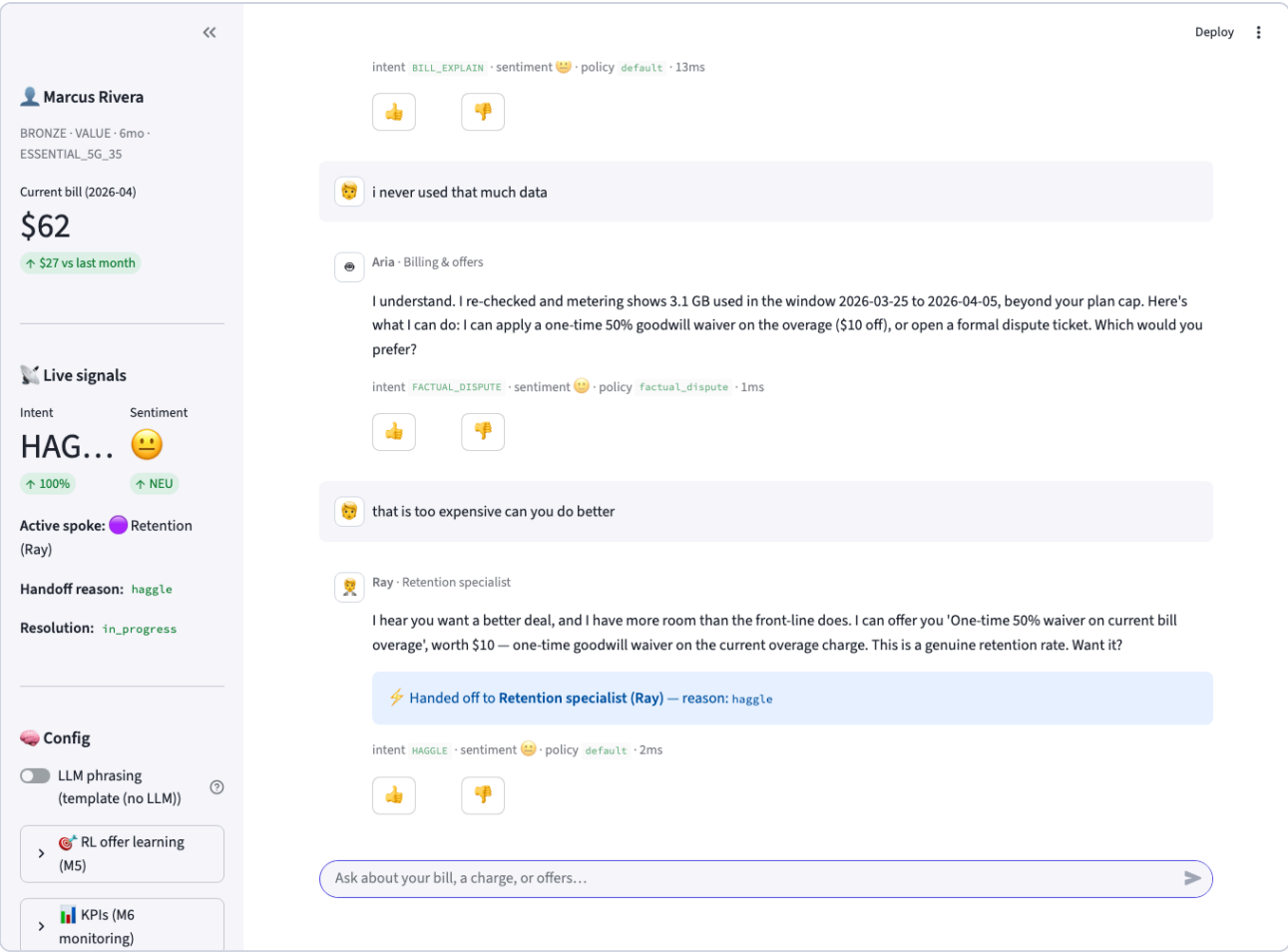


Fig 9.4 — Hub-and-spoke hand-off to the retention specialist, with the handoff badge and sidebar signals.

9.4 Discussion

The results confirm that the hybrid architecture achieves its central aim: fluent, human-like interaction with strict factual reliability. The deterministic core guarantees that resolutions are correct and policy-compliant, while the optional LLM elevates the phrasing to a natural register. The live signals make the system's reasoning transparent — an observer can watch the intent, sentiment and active agent update turn by turn — which is valuable both for demonstration and for operational trust. The reinforcement-learning loop, though modest in a short session, provides the mechanism for long-run improvement that the objectives require.

9.5 Fulfilment of Objectives

Objective	Evidence of fulfilment
1. Automated real-time NLP support	End-to-end chatbot resolving billing and offer queries in real time
2. Improve quality & efficiency	Grounded NLG + low latency; RL improves offer relevance; KPIs track outcomes
3. Deep-learning NLU & NLG	MLP neural intent + sentiment models; grounded generation with local LLM
4. Reinforcement learning from feedback	Persisted epsilon-greedy bandit updated by accept/decline and thumbs feedback
5. Evaluation with key metrics	Accuracy/precision/recall/F1 + resolution rate, latency, CSAT proxy

Table 9.2 — Mapping of Results to Objectives.

Chapter 10 — Conclusion and Future Scope

10.1 Conclusion

This project set out to design and implement an intelligent conversational AI chatbot that delivers automated, accurate and real-time customer support for telecom billing disputes and offer recommendations. The delivered system meets that goal end-to-end: it validates the customer's identity, understands free-text queries through deep-learning NLU, resolves billing disputes with evidence-backed remedies, recommends and applies personalized offers, escalates difficult cases through a hub-and-spoke hand-off to a retention specialist, phrases its replies naturally via an optional local LLM while never fabricating facts, learns which offers customers accept through a reinforcement-learning loop, and exposes live signals and KPIs for monitoring. All five objectives are satisfied, and the system runs reproducibly on commodity hardware with a single command. The central methodological contribution is the demonstration that a hybrid design — deterministic understanding and decision-making, optional generative fluency, and adaptive learning — can reconcile the fluency of modern language models with the correctness and transparency that a regulated billing domain demands.

10.2 Limitations

As a proof of concept, the system has deliberate limitations. The NLU models are trained on a compact hand-labelled corpus, capping held-out accuracy. The data adapters are synthetic rather than connected to live CRM/billing systems. The supervisor approval in the retention flow is simulated rather than routed through a real approval channel, and there is no transfer to a human agent. The system is text-only and monolingual.

10.3 Future Scope

- **Corpus expansion:** mine real, anonymized chat logs and support tickets to enlarge the labelled corpus and lift NLU accuracy substantially.
- **Transformer NLU:** replace or augment the MLP classifiers with fine-tuned transformer encoders for stronger generalization.
- **Live back-ends:** swap the synthetic adapters for connectors to production CRM, billing and ticketing systems.
- **Contextual RL:** extend the bandit to a contextual policy conditioned on customer features for sharper personalization.
- **Approval & validation workflows:** add a real supervisor approval channel and a proactive validation agent that confirms promised actions were executed, reducing repeat contacts.
- **Multilingual & voice:** extend to additional languages and to a voice channel.
- **Retrieval-augmented answers:** ground open FAQ-style questions in a knowledge base via retrieval-augmented generation.

References

1. Weizenbaum, J. (1966). ELIZA — A Computer Program for the Study of Natural Language Communication between Man and Machine. *Communications of the ACM*.
2. Jurafsky, D. & Martin, J. H. *Speech and Language Processing* (3rd ed. draft). Stanford University.
3. Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS*.
4. Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*.
5. Sutton, R. S. & Barto, A. G. *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
6. Ricci, F., Rokach, L. & Shapira, B. (eds.). *Recommender Systems Handbook*. Springer.
7. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*.
8. Hutto, C. J. & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis. *ICWSM*.
9. Streamlit Documentation. <https://docs.streamlit.io>
10. Ollama — Local Large Language Model Runtime. <https://ollama.com>
11. Young, S. et al. (2013). POMDP-based Statistical Spoken Dialogue Systems: A Review. *Proceedings of the IEEE*.
12. Auer, P., Cesa-Bianchi, N. & Fischer, P. (2002). Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning*.

Appendices

Appendix A — Sample Synthetic Data

An abbreviated example of a customer record and a bill line item with evidence, illustrating the structure used throughout the system:

```
"CUST-TELCO-1003": {
  "customer_id": "CUST-TELCO-1003", "first_name": "Marcus", "last_name": "Rivera",
  "plan": "ESSENTIAL_5G_35", "tenure_months": 6, "arpu_usd": 35,
  "loyalty_tier": "BRONZE", "segment": "VALUE", "last4_id": "1100"
}
// bill line item with evidence
{ "charge_id": "CHG-OVG-003", "description": "Data overage – 3 GB beyond plan",
  "amount_usd": 21, "type": "OVERAGE",
  "evidence": { "usage_gb": 3.1, "window": "2026-03-25 to 2026-04-05" } }
```

Appendix B — Key Code Listing (Per-Turn Entry Point)

```
def handle_turn(state, text, use_llm=True):
    frame = pipeline.understand(text)          # NLU → TurnFrame
    directive = policy_engine.evaluate(frame, state)
    if state.active_spoke == "billing":
        flow_response = _route_billing(state, frame)
        reason = _detect_handoff(frame, state, directive, flow_response)
        if reason:                             # hub-and-spoke handoff
            _do_handoff(state, reason)
            flow_response = _retention_intro(state, frame)
    else:
        flow_response = retention_flow.handle(state, frame)
    text_out = generator.compose(PERSONA[state.active_spoke],
                                flow_response["template"],
                                facts=flow_response.get("facts"),
                                directive=directive, use_llm=use_llm)
    # ... assemble response, record turn + latency, return
```

Appendix C — Milestone Outcome Summary

See Table 5.1 for the consolidated milestone outcomes. Each milestone produced a tangible, testable artefact: the synthetic dataset (M1); the labelled corpus and pre-processing (M2); the trained NLU pipeline (M3); the grounded NLG module with a pluggable LLM provider (M4); the reinforcement-learning bandit with a persisted policy (M5); and the deployed, monitored Streamlit application (M6). Collectively these artefacts constitute the complete, runnable proof-of-concept described in this report.

Appendix D — System Reference Tables

D.1 Intent Taxonomy

The fourteen intents recognized by the NLU stack, with the customer behaviour each represents:

Intent	Meaning / example
GREETING	Opening pleasantry — “hi, I need help”
BILL_EXPLAIN	Request to understand the bill — “why is my bill higher?”
BILL_DISPUTE	Assertion that a charge is wrong — “this roaming charge is incorrect”
FACTUAL_DISPUTE	Denial of the underlying fact — “I never went to Mexico”
PROMO_INQUIRY	Interest in offers — “any deals for me?”
CONFIRM_YES	Affirmation — “yes, apply it”
CONFIRM_NO	Refusal — “no thanks”
HAGGLE	Negotiation pressure — “that’s too expensive, do better”
CREATE_TICKET	Request to formalize a dispute — “open a complaint”
TICKET_STATUS	Query about an existing ticket — “any update on my dispute?”
HUMAN_HANDOFF	Request for a specialist/person — “let me talk to someone”
PAYMENT_QUERY	Payment-history question — “when did I last pay?”
GOODBYE	Closing — “that’s all, thanks”
FALLBACK	Out-of-scope or unintelligible input

D.2 Offer Catalogue

Offer	Target tier	Min tenure	Value	Applies to
Gold loyalty — 10% off for 3 months	GOLD	36 months	\$24	Plan charge
Add-on 5 GB data free for 2 months	Any	3 months	\$12	Data add-on
Upgrade to Family Share — first month free	SILVER	6 months	\$120	Plan upgrade
One-time 50% waiver on overage	BRONZE	1 month	\$10	Overage charge

D.3 Demonstration Customers

Customer	Tier	Bill	Disputable charge	Best-suited demonstration
Ethan Walker	GOLD	\$122	\$30 Mexico roaming (with evidence)	Roaming dispute → ticket; loyalty/data offers
Sarah Chen	SILVER	\$120	—	Family / data offer recommendations
Marcus Rivera	BRONZE	\$62	\$21 data overage (with metering)	Overage dispute → haggle → hand-off → close

D.4 Key Performance Indicators

KPI	Definition
Resolution rate	Share of sessions ending in a resolved or escalated-and-closed state
Average latency	Mean per-turn processing time in milliseconds
CSAT proxy	Ratio of thumbs-up to total feedback ratings
Intent accuracy	Correct-intent proportion on the held-out evaluation split
Sentiment accuracy	Correct-sentiment proportion on the held-out split

D.5 Handoff Triggers

Trigger	Condition
offers_exhausted	The offers flow has no remaining eligible offer to pitch
voice_cannot_resolve	The billing flow needs a credit beyond front-line authority
customer_requested	The customer explicitly asks for a specialist
haggle	The customer negotiates or threatens to leave
negative_sentiment	The policy engine raises escalation after sustained negative sentiment (≥ 2 turns)

— End of Report —